



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №5

«Розв'язання систем лінійних алгебраїчних рівнянь»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Порєв В. М.*

Тема: «Розв’язання систем лінійних алгебраїчних рівнянь».

Мета: Вивчити алгоритми методів розв’язання систем лінійних алгебраїчних рівнянь на ПК.

Хід роботи

У даній лабораторній роботі розглядається задача чисельного розв’язання системи лінійних алгебраїчних рівнянь методом Гауса з вибором головного елемента. За варіантом потрібно розв’язати систему

$$Ax = b,$$

де

$$A = \begin{pmatrix} 0.63 & 1.00 & 0.71 & 0.34 \\ 1.17 & 0.18 & -0.65 & 0.71 \\ 2.71 & -0.75 & 1.17 & -2.35 \\ 3.58 & 0.21 & -3.45 & -1.18 \end{pmatrix}, \quad b = \begin{pmatrix} 2.08 \\ 0.17 \\ 1.28 \\ 0.05 \end{pmatrix}.$$

Порядок виконання роботи:

- Ознайомитись із методикою розв’язання СЛАР на ПК.
- Скласти розширену матрицю системи.
- Реалізувати прямий хід методу Гауса з вибором головного елемента за стовпцем.
- Виконати зворотний хід і знайти вектор невідомих.
- Перевірити результат за нев’язкою $r = Ax - b$.
- Розробити графічну програму для введення системи, запуску обчислень, перегляду кроків методу та збереження звіту.

Теоретичні відомості:

Метод Гауса зводить початкову систему до еквівалентної верхньотрикутної системи. На k -му кроці серед рядків $k, k + 1, \dots, n$ вибирається рядок із найбільшим за модулем елементом у поточному стовпці:

$$p = \arg \max_{i=k, \dots, n} |a_{ik}|.$$

Якщо $p \neq k$, рядки p та k міняються місцями. Далі для всіх рядків нижче головного обчислюються множники

$$m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad i = k + 1, \dots, n,$$

і виконується занулення:

$$a_{ij}^{new} = a_{ij} - m_{ik}a_{kj}, \quad b_i^{new} = b_i - m_{ik}b_k.$$

Після завершення прямого ходу невідомі знаходяться зворотним ходом:

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}, \quad i = n, n-1, \dots, 1.$$

Початкова розширена матриця:

$$\left[\begin{array}{cccc|c} 0.63 & 1.00 & 0.71 & 0.34 & 2.08 \\ 1.17 & 0.18 & -0.65 & 0.71 & 0.17 \\ 2.71 & -0.75 & 1.17 & -2.35 & 1.28 \\ 3.58 & 0.21 & -3.45 & -1.18 & 0.05 \end{array} \right].$$

На першому кроці найбільший за модулем елемент першого стовпця дорівнює 3.58, тому виконується перестановка $R_1 \leftrightarrow R_4$. Множники занулення:

$$m_{21} = 0.3268156425, \quad m_{31} = 0.7569832402, \quad m_{41} = 0.1759776536.$$

Після першого кроку:

$$\left[\begin{array}{cccc|c} 3.58 & 0.21 & -3.45 & -1.18 & 0.05 \\ 0 & 0.111369 & 0.477514 & 1.09564 & 0.153659 \\ 0 & -0.908966 & 3.78159 & -1.45676 & 1.24215 \\ 0 & 0.963045 & 1.31712 & 0.547654 & 2.07120 \end{array} \right].$$

На другому кроці у другому стовпці вибирається рядок R_4 , оскільки $|0.963045|$ є найбільшим серед доступних елементів. Виконується перестановка $R_2 \leftrightarrow R_4$. Множники:

$$m_{32} = -0.9438466224, \quad m_{42} = 0.1156423123.$$

Після другого кроку:

$$\left[\begin{array}{cccc|c} 3.58 & 0.21 & -3.45 & -1.18 & 0.05 \\ 0 & 0.963045 & 1.31712 & 0.547654 & 2.07120 \\ 0 & 0 & 5.02475 & -0.939859 & 3.19705 \\ 0 & 0 & 0.325199 & 1.03231 & -0.0858593 \end{array} \right].$$

На третьому кроці головний елемент дорівнює 5.02475418395, перестановка рядків не потрібна. Множник занулення:

$$m_{43} = 0.06471935070.$$

Отримана верхньотрикутна система:

$$\left[\begin{array}{cccc|c} 3.58 & 0.21 & -3.45 & -1.18 & 0.05 \\ 0 & 0.963045 & 1.31712 & 0.547654 & 2.07120 \\ 0 & 0 & 5.02475 & -0.939859 & 3.19705 \\ 0 & 0 & 0 & 1.09314 & -0.292770 \end{array} \right].$$

Зворотний хід:

$$\begin{aligned} x_4 &= \frac{-0.292770075581}{1.09313757367} = -0.267825461892, \\ x_3 &= \frac{2.94532891398}{5.02475418395} = 0.586163781582, \\ x_2 &= \frac{1.44582696126}{0.963044692737} = 1.50130826966, \\ x_1 &= \frac{1.44095626480}{3.58} = 0.402501749943. \end{aligned}$$

Отже, розв'язок системи:

$$x = \begin{pmatrix} 0.402501749943 \\ 1.501308269656 \\ 0.586163781582 \\ -0.267825461892 \end{pmatrix}.$$

У форматі з округленням до чотирьох знаків:

$$x_1 = 0.4025, \quad x_2 = 1.5013, \quad x_3 = 0.5862, \quad x_4 = -0.2678.$$

Визначник матриці:

$$\det(A) = 18.93734589.$$

Перевірка результату:

Для контролю точності було обчислено вектор нев'язки

$$r = Ax - b.$$

Отримані значення:

$$r = \begin{pmatrix} 0.000000000000 \cdot 10^0 \\ 5.551115123126 \cdot 10^{-17} \\ -2.220446049250 \cdot 10^{-16} \\ -1.387778780781 \cdot 10^{-17} \end{pmatrix},$$
$$\max |r_i| = 2.220446049250 \cdot 10^{-16}.$$

Значення нев'язки близьке до машинного нуля, тому знайдений розв'язок є коректним.

Аналіз результатів:

Отримані результати дають змогу оцінити не лише сам розв'язок, а й якість виконаного чисельного процесу. Початкова матриця не є діагонально домінантною в усіх рядках, тому застосування звичайного методу Гауса без вибору головного елемента могло б призвести до більшого накопичення похибок. Саме тому в реалізації використано частковий вибір головного елемента за стовпцем.

На першому кроці найбільшим за модулем елементом першого стовпця є 3.58, тому четвертий рядок було перенесено на позицію першого. На другому кроці найбільшим кандидатом у другому стовпці став елемент 0.963045, тому знову виконано перестановку рядків. Третій крок уже не потребував перестановки, оскільки елемент 5.02475418395 був достатнім головним елементом. Така послідовність дій показує, що вибір головного елемента реально вплинув на хід алгоритму, а не був формальною операцією.

Після прямого ходу отримано верхньотрикутну матрицю з такими діагональними елементами:

$$3.58, \quad 0.963045, \quad 5.02475, \quad 1.09314.$$

Жоден із них не є нульовим або близьким до нуля, тому зворотний хід виконується коректно. Це також узгоджується з обчисленням визначником:

$$\det(A) = 18.93734589 \neq 0.$$

Ненульовий визначник означає, що система має єдиний розв'язок.

Для оцінки точності було використано вектор нев'язки $r = Ax - b$. Його компоненти мають порядок 10^{-16} , а максимальна абсолютна нев'язка становить

$$\max |r_i| = 2.220446049250 \cdot 10^{-16}.$$

Це значення фактично відповідає машинній похибці для обчислень із плаваючою комою подвійної точності. Отже, після підстановки знайденого вектора x у початкову систему ліва частина практично повністю збігається з вектором правих частин.

Показник	Значення	Інтерпретація
$\det(A)$	18.93734589	система має єдиний розв'язок
$\max r_i $	$2.220446049250 \cdot 10^{-16}$	похибка на рівні машинної точності
Кількість перестановок	2	pivoting був потрібний для стійкості
Кількість кроків прямого ходу	3	відповідає системі розміру 4×4

Остаточний розв'язок

$$x = (0.402501749943; 1.501308269656; 0.586163781582; -0.267825461892)$$

після округлення до чотирьох знаків дає значення

$$x_1 = 0.4025, \quad x_2 = 1.5013, \quad x_3 = 0.5862, \quad x_4 = -0.2678,$$

що збігається з контрольними значеннями варіанта. Таким чином, розроблена програма правильно реалізує метод Гауса з вибором головного елемента, а отримані числові результати є достовірними.

Блок-схема алгоритму:

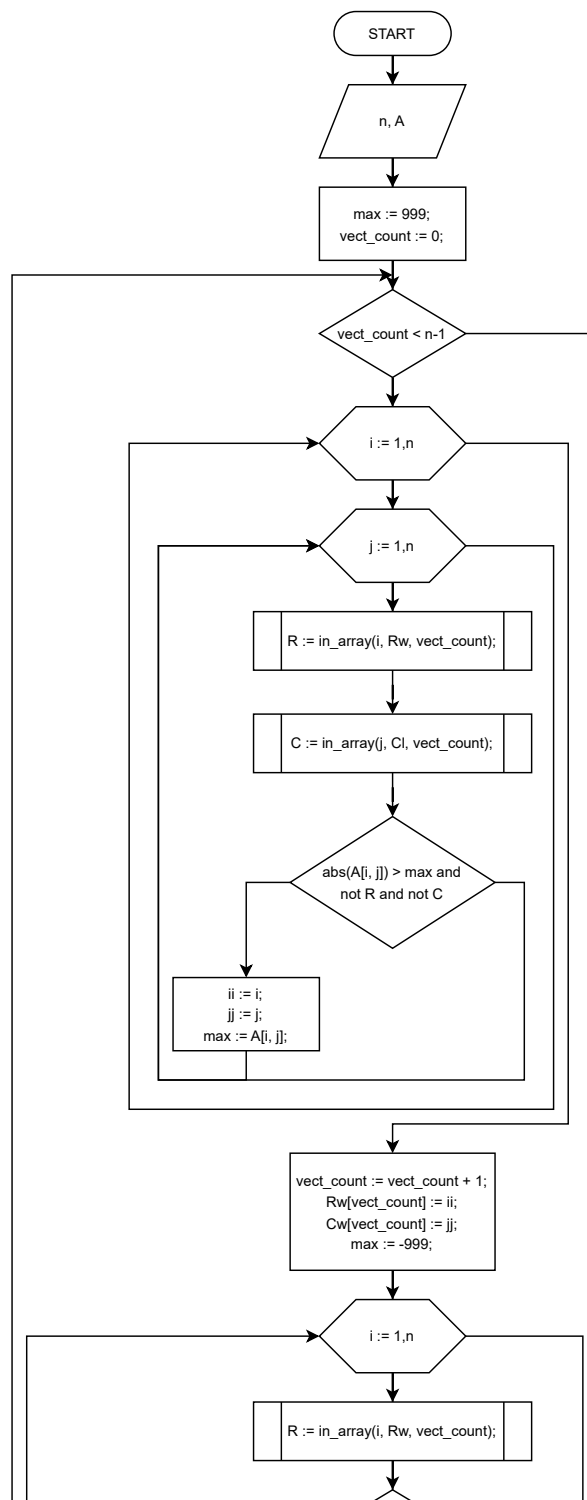


Рис. 1: Блок-схема алгоритму, сторінка 1

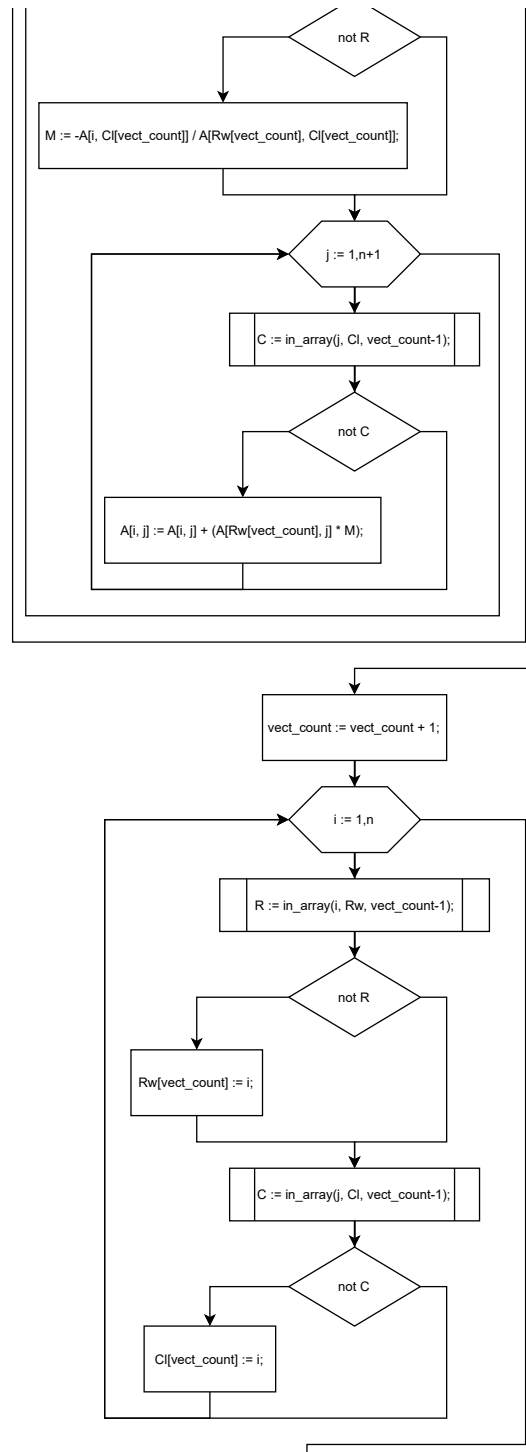


Рис. 2: Блок-схема алгоритму, сторінка 2

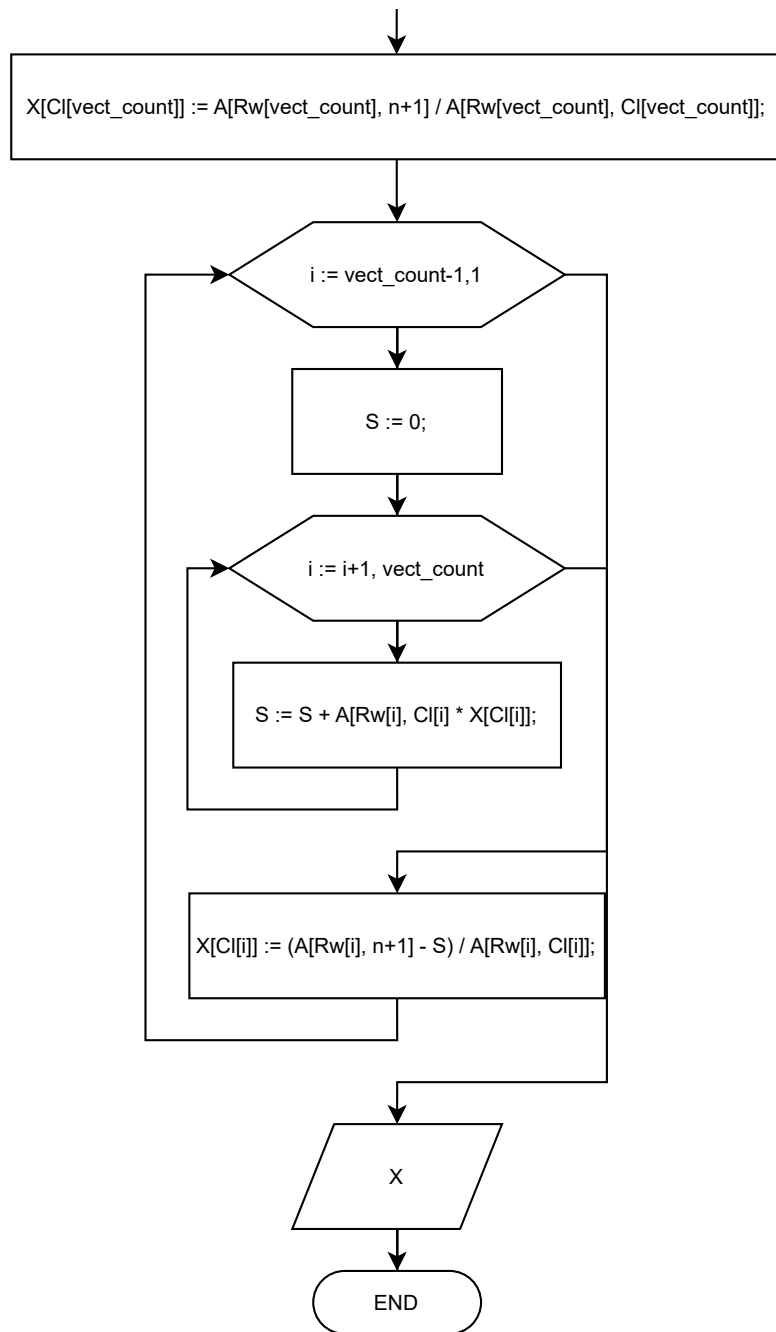


Рис. 3: Блок-схема алгоритму, сторінка 3

Опис програмної реалізації:

Програма написана мовою Python із розділенням відповідальностей між модулями. Точка входу розташована у файлі `app.py`, а вихідний код програми розміщено в каталозі `src`. Така структура відокремлює графічний інтерфейс, алгоритм, моделі даних, роботу з JSON-файлами та формування звіту.

Фактична структура проекту:

```
project/
|-- app.py
|-- src/
|   |-- __init__.py
|   |-- gaussian.py
|   |-- gui.py
|   |-- io_utils.py
|   |-- models.py
|   |-- report.py
|   |-- theme.py
|-- test_json/
    |-- variant_from_photo.json
```

`gaussian.py` містить чисельний алгоритм, `models.py` — структури даних, `io_utils.py` — завантаження JSON і збереження звітів, `report.py` — формування текстового звіту, а `gui.py` — графічний інтерфейс користувача.

У програмі реалізовано:

- введення та редагування матриці коефіцієнтів;
- вибір розмірності системи від 2×2 до 10×10 ;
- завантаження початкових даних із JSON-файлу;
- розв’язання системи методом Гауса з частковим вибором головного елемента;
- перегляд кроків прямого ходу;
- перегляд вектора розв’язку та нев’язок;
- збереження текстового звіту;
- окрему вкладку Додатково з кнопкою Запустити ефект;
- кнопку Намалювати Гауса, яка виводить на полотно reference-портрет Гауса;
- кнопку Рожевий режим, яка переводить увесь інтерфейс у декоративну рожеву тему;
- кнопку Грати музику, яка запускає локальний MP3-файл із каталогу `assets`.

Формат JSON-файлу:

```
1 {
2   "name": "Варіант з фото",
3   "matrix": [
4     [0.63, 1.00, 0.71, 0.34],
5     [1.17, 0.18, -0.65, 0.71],
6     [2.71, -0.75, 1.17, -2.35],
7     [3.58, 0.21, -3.45, -1.18]
8   ],
9   "vector": [2.08, 0.17, 1.28, 0.05]
10 }
```

Лістинг основних модулів:

Файл app.py:

```
1 from src.gui import main
2
3
4 if __name__ == "__main__":
5     main()
```

Файл src/gaussian.py:

```
1 from __future__ import annotations
2
3 from math import isfinite
4
5 from src.models import (
6     BackSubstitutionStep,
7     EliminationStep,
8     GaussianResult,
9     MatrixProblem,
10    NumberMatrix,
11    NumberVector,
12    RowOperation,
13 )
14
15
16 DEFAULT_PROBLEM = MatrixProblem(
17     name="Варіант з фото",
18     matrix=[
19         [0.63, 1.00, 0.71, 0.34],
20         [1.17, 0.18, -0.65, 0.71],
21         [2.71, -0.75, 1.17, -2.35],
22         [3.58, 0.21, -3.45, -1.18],
23     ],
24     vector=[2.08, 0.17, 1.28, 0.05],
25 )
26
27
28 def build_default_problem() -> MatrixProblem:
29     return MatrixProblem(
30         name=DEFAULT_PROBLEM.name,
31         matrix=clone_matrix(DEFAULT_PROBLEM.matrix),
32         vector=DEFAULT_PROBLEM.vector[:],
33     )
34
35
36 def clone_matrix(matrix: NumberMatrix) -> NumberMatrix:
37     return [row[:] for row in matrix]
38
39
40 def validate_problem(problem: MatrixProblem) -> None:
41     size = problem.size
42     if size < 2:
43         raise ValueError("Система має містити щонайменше 2 рівняння.")
```

```

44     if size > 10:
45         raise ValueError("Для зручного GUI підтримується розмір до 10.")
46     if len(problem.matrix) != size:
47         raise ValueError("Кількість рядків матриці має збігатися з розміром вектора
48             ↪ b.")
49
50     for row_index, row in enumerate(problem.matrix, start=1):
51         if len(row) != size:
52             raise ValueError(f"Рядок {row_index} матриці має містити {size}
53                 ↪ коефіцієнтів.")
54         for value in row:
55             if not isfinite(value):
56                 raise ValueError("Усі коефіцієнти мають бути скінченними числами.")
57
58     for value in problem.vector:
59         if not isfinite(value):
60             raise ValueError("Вектор вільних членів має містити скінченні числа.")
61
62 def solve_gauss_partial_pivot(
63     problem: MatrixProblem,
64     tolerance: float = 1e-12,
65 ) -> GaussianResult:
66     validate_problem(problem)
67     if tolerance <= 0:
68         raise ValueError("Допуск має бути додатним.")
69
70     size = problem.size
71     matrix = clone_matrix(problem.matrix)
72     vector = problem.vector[:]
73     determinant_sign = 1.0
74     elimination_steps: list[EliminationStep] = []
75
76     for column in range(size - 1):
77         selected_row = max(range(column, size), key=lambda row:
78             ↪ abs(matrix[row][column]))
79         pivot_value = matrix[selected_row][column]
80         if abs(pivot_value) <= tolerance:
81             raise ValueError(
82                 f"Система вироджена або майже вироджена: "
83                 f"нульовий головний елемент у стовпці {column + 1}."
84             )
85
86         if selected_row != column:
87             matrix[column], matrix[selected_row] = matrix[selected_row],
88                 ↪ matrix[column]
89             vector[column], vector[selected_row] = vector[selected_row],
90                 ↪ vector[column]
91             determinant_sign *= -1.0
92
93     operations: list[RowOperation] = []
94     pivot_value = matrix[column][column]
95
96     for row in range(column + 1, size):

```

```

93         factor = matrix[row][column] / pivot_value
94         operations.append(RowOperation(row, column, factor))
95         matrix[row][column] = 0.0
96
97         for inner_column in range(column + 1, size):
98             matrix[row][inner_column] -= factor * matrix[column][inner_column]
99             if abs(matrix[row][inner_column]) < tolerance:
100                 matrix[row][inner_column] = 0.0
101
102         vector[row] -= factor * vector[column]
103         if abs(vector[row]) < tolerance:
104             vector[row] = 0.0
105
106         elimination_steps.append(
107             EliminationStep(
108                 step=column + 1,
109                 pivot_column=column,
110                 selected_row=selected_row,
111                 pivot_row=column,
112                 pivot_value=pivot_value,
113                 matrix=clone_matrix(matrix),
114                 vector=vector[:],
115                 operations=operations,
116             )
117         )
118
119     last_pivot = matrix[-1][-1]
120     if abs(last_pivot) <= tolerance:
121         raise ValueError("Система не має єдиного розв'язку: останній діагональний
122             ↪ елемент нульовий.")
123
124     solution = [0.0 for _ in range(size)]
125     back_steps: list[BackSubstitutionStep] = []
126     for row in range(size - 1, -1, -1):
127         known_sum = sum(matrix[row][column] * solution[column] for column in
128             ↪ range(row + 1, size))
129         numerator = vector[row] - known_sum
130         denominator = matrix[row][row]
131         if abs(denominator) <= tolerance:
132             raise ValueError(f"Неможливо виконати зворотний хід у рядку {row + 1}.")
133
134         solution[row] = numerator / denominator
135         back_steps.append(
136             BackSubstitutionStep(
137                 row=row,
138                 numerator=numerator,
139                 denominator=denominator,
140                 value=solution[row],
141             )
142         )
143
144     determinant = determinant_sign
145     for diagonal_index in range(size):
146         determinant *= matrix[diagonal_index][diagonal_index]

```

```

145
146     residuals = compute_residuals(problem.matrix, solution, problem.vector)
147     residual_norm = max((abs(value) for value in residuals), default=0.0)
148
149     return GaussianResult(
150         problem=problem,
151         solution=solution,
152         determinant=determinant,
153         residuals=residuals,
154         residual_norm=residual_norm,
155         elimination_steps=elimination_steps,
156         back_steps=list(reversed(back_steps)),
157         upper_matrix=clone_matrix(matrix),
158         transformed_vector=vector[:],
159     )
160
161
162 def compute_residuals(
163     matrix: NumberMatrix,
164     solution: NumberVector,
165     vector: NumberVector,
166 ) -> NumberVector:
167     residuals: NumberVector = []
168     for row, right_side in zip(matrix, vector):
169         actual = sum(coefficient * value for coefficient, value in zip(row,
170             ↪ solution))
171         residuals.append(actual - right_side)
172     return residuals
173
174 def rounded_solution(solution: NumberVector, digits: int = 4) -> NumberVector:
175     return [round(value, digits) for value in solution]

```

Файл src/models.py:

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4
5
6 NumberMatrix = list[list[float]]
7 NumberVector = list[float]
8
9
10 @dataclass(frozen=True)
11 class PageConfig:
12     page_id: str
13     nav_title: str
14     title: str
15     subtitle: str
16
17
18 @dataclass(frozen=True)
19 class MatrixProblem:

```

```

20     name: str
21     matrix: NumberMatrix
22     vector: NumberVector
23
24     @property
25     def size(self) -> int:
26         return len(self.vector)
27
28
29 @dataclass(frozen=True)
30 class RowOperation:
31     target_row: int
32     pivot_row: int
33     factor: float
34
35     def as_text(self) -> str:
36         return (
37             f"R{self.target_row + 1} <- R{self.target_row + 1} - "
38             f"({self.factor:.10g}) * R{self.pivot_row + 1}"
39         )
40
41
42 @dataclass(frozen=True)
43 class EliminationStep:
44     step: int
45     pivot_column: int
46     selected_row: int
47     pivot_row: int
48     pivot_value: float
49     matrix: NumberMatrix
50     vector: NumberVector
51     operations: list[RowOperation] = field(default_factory=list)
52
53     @property
54     def did_swap(self) -> bool:
55         return self.selected_row != self.pivot_row
56
57
58 @dataclass(frozen=True)
59 class BackSubstitutionStep:
60     row: int
61     numerator: float
62     denominator: float
63     value: float
64
65
66 @dataclass(frozen=True)
67 class GaussianResult:
68     problem: MatrixProblem
69     solution: NumberVector
70     determinant: float
71     residuals: NumberVector
72     residual_norm: float
73     elimination_steps: list[EliminationStep]

```

```

74     back_steps: list[BackSubstitutionStep]
75     upper_matrix: NumberMatrix
76     transformed_vector: NumberVector

```

Файл src/io_utils.py:

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5  from typing import Any
6
7  from src.models import MatrixProblem, NumberMatrix, NumberVector
8
9
10 def load_system_config(path: str) -> MatrixProblem:
11     with Path(path).open("r", encoding="utf-8") as file:
12         data = json.load(file)
13
14     if not isinstance(data, dict):
15         raise ValueError("JSON-файл має містити об'єкт.")
16
17     matrix_source = _first_present(data, "matrix", "coefficients", "a")
18     vector_source = _first_present(data, "vector", "rhs", "b")
19     name = str(data.get("name", "Система з JSON"))
20
21     matrix = _parse_matrix(matrix_source)
22     vector = _parse_vector(vector_source)
23     return MatrixProblem(name=name, matrix=matrix, vector=vector)
24
25
26 def save_text_report(path: str, report: str) -> None:
27     Path(path).write_text(report, encoding="utf-8")
28
29
30 def _first_present(data: dict[str, Any], *keys: str) -> Any:
31     for key in keys:
32         if key in data:
33             return data[key]
34     raise ValueError(f"JSON має містити одне з полів: {'', ' '.join(keys)}.")
35
36
37 def _parse_matrix(value: Any) -> NumberMatrix:
38     if not isinstance(value, list) or not value:
39         raise ValueError("Матриця має бути непорожнім двовимірним масивом.")
40
41     matrix: NumberMatrix = []
42     for row in value:
43         if not isinstance(row, list) or not row:
44             raise ValueError("Кожен рядок матриці має бути непорожнім масивом.")
45         matrix.append([_parse_number(item) for item in row])
46     return matrix
47
48

```

```

49 def _parse_vector(value: Any) -> NumberVector:
50     if not isinstance(value, list) or not value:
51         raise ValueError("Вектор b має бути непорожнім масивом.")
52     return [_parse_number(item) for item in value]
53
54
55 def _parse_number(value: Any) -> float:
56     if isinstance(value, bool):
57         raise ValueError("Логічні значення не є числами для СЛАР.")
58     if isinstance(value, int | float):
59         return float(value)
60     if isinstance(value, str):
61         normalized = value.strip().replace(",", ".")
62         if not normalized:
63             raise ValueError("Порожній рядок не можна перетворити на число.")
64         return float(normalized)
65     raise ValueError(f"Непідтримуване числове значення: {value!r}.")

```

Файл src/report.py:

```

1  from __future__ import annotations
2
3  from src.models import EliminationStep, GaussianResult, MatrixProblem, NumberMatrix,
   ↪  NumberVector
4
5
6  def build_short_result(result: GaussianResult) -> str:
7      lines = ["Розв'язок:"]
8      for index, value in enumerate(result.solution, start=1):
9          lines.append(f"x{index} = {value:.10f}")
10
11     lines.append(f"det(A) = {result.determinant:.10g}")
12     lines.append(f"max |Ax - b| = {result.residual_norm:.3e}")
13     return "\n".join(lines)
14
15
16 def build_full_report(result: GaussianResult) -> str:
17     problem = result.problem
18     lines: list[str] = [
19         "Лабораторна робота N5",
20         "Тема: Розв'язання систем лінійних алгебраїчних рівнянь",
21         "",
22         "Метод: метод Гауса з вибором головного елемента.",
23         f"Система: {problem.name}",
24         f"Розмірність: {problem.size} x {problem.size}",
25         "",
26         "Початкова розширена матриця:",
27         *_format_augmented(problem.matrix, problem.vector),
28         "",
29         "Прямий хід:",
30     ]
31
32     for step in result.elimination_steps:
33         lines.extend(_format_step(step))

```

```

34
35 lines.extend(
36     [
37         "",
38         "Верхньотрикутна система:",
39         *_format_augmented(result.upper_matrix, result.transformed_vector),
40         "",
41         "Зворотний хід:",
42     ]
43 )
44
45 for step in result.back_steps:
46     lines.append(
47         f"x{step.row + 1} = {step.numerator:.12g} / "
48         f"{step.denominator:.12g} = {step.value:.12g}"
49     )
50
51 lines.extend(["", "Розв'язок:"])
52 for index, value in enumerate(result.solution, start=1):
53     lines.append(f"x{index} = {value:.12g}")
54
55 lines.extend(
56     [
57         "",
58         "Перевірка Ax - b:",
59     ]
60 )
61 for index, value in enumerate(result.residuals, start=1):
62     lines.append(f"r{index} = {value:.12e}")
63
64 lines.extend(
65     [
66         "",
67         f"max |Ax - b| = {result.residual_norm:.12e}",
68         f"det(A) = {result.determinant:.12g}",
69     ]
70 )
71 return "\n".join(lines)
72
73
74 def format_matrix_for_display(matrix: NumberMatrix, vector: NumberVector | None =
↪ None) -> str:
75     return "\n".join(_format_augmented(matrix, vector) if vector is not None else
↪ _format_matrix(matrix))
76
77
78 def _format_step(step: EliminationStep) -> list[str]:
79     lines = [
80         "",
81         f"Крок {step.step}. Стовпець {step.pivot_column + 1}; "
82         f"вибраний рядок R{step.selected_row + 1}; "
83         f"головний елемент = {step.pivot_value:.12g}.",
84     ]
85     if step.did_swap:

```

```

86         lines.append(f"Перестановка: R{step.pivot_row + 1} <-> R{step.selected_row +
↪ 1}.")
87     else:
88         lines.append("Перестановка рядків не потрібна.")
89
90     if step.operations:
91         lines.append("Операції:")
92         lines.extend(f" {operation.as_text()}" for operation in step.operations)
93     else:
94         lines.append("Операцій занулення немає.")
95
96     lines.append("Матриця після кроку:")
97     lines.extend(_format_augmented(step.matrix, step.vector))
98     return lines
99
100
101 def _format_augmented(matrix: NumberMatrix, vector: NumberVector | None) ->
↪ list[str]:
102     rows: list[str] = []
103     for row_index, row in enumerate(matrix):
104         coefficients = " ".join(f"{value:>13.6g}" for value in row)
105         if vector is None:
106             rows.append(f"[ {coefficients} ]")
107         else:
108             rows.append(f"[ {coefficients} | {vector[row_index]:>13.6g} ]")
109     return rows
110
111
112 def _format_matrix(matrix: NumberMatrix) -> list[str]:
113     return _format_augmented(matrix, None)

```

Файл src/theme.py:

```

1 from __future__ import annotations
2
3
4 DEFAULT_THEME: dict[str, str] = {
5     "bg": "#0D0D0D",
6     "topbar": "#111111",
7     "panel": "#171717",
8     "panel_alt": "#1D1D1D",
9     "surface": "#101010",
10    "surface_soft": "#2C2C2C",
11    "surface_strong": "#3A3A3A",
12    "segment": "#242424",
13    "segment_hover": "#313131",
14    "segment_active": "#1F8A63",
15    "accent": "#37C38A",
16    "accent_hover": "#2FA675",
17    "accent_text": "#06130D",
18    "secondary": "#4CC9F0",
19    "warning": "#F4C95D",
20    "error": "#F07167",
21    "text": "#F4F6F8",

```

```

22     "muted": "#A8B0BA",
23     "subtle": "#757D86",
24 }
25
26
27 PINK_THEME: dict[str, str] = {
28     "bg": "#FFF1FA",
29     "topbar": "#FFD3EA",
30     "panel": "#FFE3F2",
31     "panel_alt": "#FFF8FC",
32     "surface": "#FFFFFF",
33     "surface_soft": "#F3A4CB",
34     "surface_strong": "#E6579F",
35     "segment": "#F8B8D7",
36     "segment_hover": "#F29AC5",
37     "segment_active": "#FF63AE",
38     "accent": "#FF6DB8",
39     "accent_hover": "#E9509E",
40     "accent_text": "#3B0623",
41     "secondary": "#64C9FF",
42     "warning": "#FFD166",
43     "error": "#FF5E8E",
44     "text": "#381029",
45     "muted": "#7A3A61",
46     "subtle": "#B56D96",
47 }
48
49
50 MAIN_THEME = DEFAULT_THEME

```

Файл src/gui.py:

```

1  from __future__ import annotations
2
3  import random
4  import base64
5  import subprocess
6  import tkinter as tk
7  from dataclasses import dataclass, field
8  from pathlib import Path
9  from tkinter import filedialog, ttk
10 from typing import Callable
11
12 import customtkinter as ctk
13 from PIL import Image, ImageTk
14
15 from src.gaussian import build_default_problem, solve_gauss_partial_pivot,
16     ↪ validate_problem
17 from src.io_utils import load_system_config, save_text_report
18 from src.models import EliminationStep, GaussianResult, MatrixProblem, PageConfig,
19     ↪ NumberMatrix, NumberVector
20 from src.report import build_full_report, build_short_result,
21     ↪ format_matrix_for_display
22 from src.theme import DEFAULT_THEME, MAIN_THEME, PINK_THEME

```

```

20
21
22 PROJECT_ROOT = Path(__file__).resolve().parents[1]
23 GAUSS_REFERENCE_PATH = PROJECT_ROOT / "assets" / "gauss_reference.jpeg"
24 MUSIC_CANDIDATES = (
25     PROJECT_ROOT / "assets.mp3",
26     PROJECT_ROOT / "assets" / "assets.mp3",
27     PROJECT_ROOT / "assets" / "Something - Remastered 2009 - The Beatles.mp3",
28 )
29
30
31 @dataclass
32 class LinearSystemState:
33     problem: MatrixProblem = field(default_factory=build_default_problem)
34     result: GaussianResult | None = None
35     report: str = ""
36
37
38 @dataclass
39 class EffectParticle:
40     x: float
41     y: float
42     dx: float
43     dy: float
44     radius: float
45     color: str
46     symbol: str
47
48
49 class BasePage(ctk.CTkFrame):
50     def __init__(
51         self,
52         master: ctk.CTkFrame,
53         page_config: PageConfig,
54         fonts: dict[str, ctk.CTkFont],
55     ) -> None:
56         super().__init__(master, corner_radius=0)
57         self.page_config = page_config
58         self.fonts = fonts
59         self.palette = MAIN_THEME
60         self.grid_columnconfigure(0, weight=1)
61
62     def refresh(self) -> None:
63         pass
64
65     def apply_theme(self, palette: dict[str, str]) -> None:
66         self.palette = palette
67         self.configure(fg_color=palette["bg"])
68
69
70 class MatrixPage(BasePage):
71     def __init__(
72         self,
73         master: ctk.CTkFrame,

```

```

74     page_config: PageConfig,
75     fonts: dict[str, ctk.CTkFont],
76     app_state: LinearSystemState,
77     on_result_changed: Callable[[], None],
78 ) -> None:
79     super().__init__(master, page_config, fonts)
80     self.app_state = app_state
81     self.on_result_changed = on_result_changed
82
83     self.status_var = tk.StringVar(value="Готово до обчислень")
84     self.error_var = tk.StringVar(value="")
85     self.size_var = tk.StringVar(value=str(self.app_state.problem.size))
86     self.matrix_entries: list[list[ctk.CTkEntry]] = []
87     self.vector_entries: list[ctk.CTkEntry] = []
88
89     self.grid_rowconfigure(1, weight=1)
90     self._build_header()
91     self._build_workspace()
92     self._set_problem(self.app_state.problem, reset_output=True)
93
94 def _build_header(self) -> None:
95     self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
96     self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
97     self.header_card.grid_columnconfigure(0, weight=1)
98
99     self.title_label = ctk.CTkLabel(
100         self.header_card,
101         text=self.page_config.title,
102         anchor="w",
103         font=self.fonts["title"],
104     )
105     self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
106
107     self.subtitle_label = ctk.CTkLabel(
108         self.header_card,
109         text=self.page_config.subtitle,
110         anchor="w",
111         justify="left",
112         wraplength=1020,
113         font=self.fonts["subtitle"],
114     )
115     self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
↵ 14))
116
117 def _build_workspace(self) -> None:
118     self.workspace = ctk.CTkFrame(self, fg_color="transparent")
119     self.workspace.grid(row=1, column=0, sticky="nsew")
120     self.workspace.grid_columnconfigure(0, weight=5)
121     self.workspace.grid_columnconfigure(1, weight=3)
122     self.workspace.grid_rowconfigure(0, weight=1)
123
124     self.input_card = ctk.CTkFrame(self.workspace, corner_radius=8,
↵ border_width=1)
125     self.input_card.grid(row=0, column=0, sticky="nsew", padx=(0, 10))

```

```

126     self.input_card.grid_columnconfigure(0, weight=1)
127     self.input_card.grid_rowconfigure(2, weight=1)
128
129     self.result_card = ctk.CTkFrame(self.workspace, corner_radius=8,
130     ↪ border_width=1)
131     self.result_card.grid(row=0, column=1, sticky="nsew", padx=(10, 0))
132     self.result_card.grid_columnconfigure(0, weight=1)
133     self.result_card.grid_rowconfigure(5, weight=1)
134
135     self._build_problem_controls()
136     self._build_matrix_area()
137     self._build_action_bar()
138     self._build_result_panel()
139
140     def _build_problem_controls(self) -> None:
141         self.controls_frame = ctk.CTkFrame(self.input_card, fg_color="transparent")
142         self.controls_frame.grid(row=0, column=0, sticky="ew", padx=20, pady=(18,
143     ↪ 8))
144         self.controls_frame.grid_columnconfigure(0, weight=1)
145
146         self.name_label = ctk.CTkLabel(
147             self.controls_frame,
148             text="Назва системи",
149             anchor="w",
150             font=self.fonts["label"],
151         )
152         self.name_label.grid(row=0, column=0, sticky="w", pady=(0, 6))
153
154         self.name_entry = ctk.CTkEntry(
155             self.controls_frame,
156             height=40,
157             corner_radius=8,
158             font=self.fonts["body"],
159         )
160         self.name_entry.grid(row=1, column=0, sticky="ew", padx=(0, 12))
161         self.name_entry.bind("<KeyRelease>", self._mark_dirty)
162
163         self.size_label = ctk.CTkLabel(
164             self.controls_frame,
165             text="Розмір",
166             anchor="w",
167             font=self.fonts["label"],
168         )
169         self.size_label.grid(row=0, column=1, sticky="w", pady=(0, 6))
170
171         self.size_menu = ctk.CTkOptionMenu(
172             self.controls_frame,
173             values=[str(value) for value in range(2, 11)],
174             variable=self.size_var,
175             width=110,
176             height=40,
177             corner_radius=8,
178             font=self.fonts["body"],
179             command=self._resize_matrix,

```

```

178     )
179     self.size_menu.grid(row=1, column=1, sticky="ew")
180
181     self.method_label = ctk.CTkLabel(
182         self.input_card,
183         text="A · x = b, метод Гауса з вибором головного елемента за стовпцем",
184         anchor="w",
185         font=self.fonts["label"],
186     )
187     self.method_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
188
189     def _build_matrix_area(self) -> None:
190         self.matrix_area = ctk.CTkScrollableFrame(
191             self.input_card,
192             corner_radius=8,
193             border_width=1,
194         )
195         self.matrix_area.grid(row=2, column=0, sticky="nsew", padx=20, pady=(0, 14))
196         self.matrix_area.grid_columnconfigure(0, weight=1)
197
198     def _build_action_bar(self) -> None:
199         self.action_bar = ctk.CTkFrame(self.input_card, fg_color="transparent")
200         self.action_bar.grid(row=3, column=0, sticky="ew", padx=20, pady=(0, 18))
201         self.action_bar.grid_columnconfigure(tuple(range(5)), weight=1)
202
203         self.solve_button = ctk.CTkButton(
204             self.action_bar,
205             text="Розв'язати",
206             height=40,
207             corner_radius=8,
208             font=self.fonts["button"],
209             command=self.solve,
210         )
211         self.solve_button.grid(row=0, column=0, sticky="ew", padx=(0, 6))
212
213         self.load_button = ctk.CTkButton(
214             self.action_bar,
215             text="JSON",
216             height=40,
217             corner_radius=8,
218             font=self.fonts["button"],
219             command=self.load_json,
220         )
221         self.load_button.grid(row=0, column=1, sticky="ew", padx=6)
222
223         self.save_button = ctk.CTkButton(
224             self.action_bar,
225             text="3Bit",
226             height=40,
227             corner_radius=8,
228             font=self.fonts["button"],
229             command=self.save_report,
230         )
231         self.save_button.grid(row=0, column=2, sticky="ew", padx=6)

```

```

232
233     self.default_button = ctk.CTkButton(
234         self.action_bar,
235         text="Вариант",
236         height=40,
237         corner_radius=8,
238         font=self.fonts["button"],
239         command=self.reset_variant,
240     )
241     self.default_button.grid(row=0, column=3, sticky="ew", padx=6)
242
243     self.clear_button = ctk.CTkButton(
244         self.action_bar,
245         text="Очистить",
246         height=40,
247         corner_radius=8,
248         font=self.fonts["button"],
249         command=self.clear_matrix,
250     )
251     self.clear_button.grid(row=0, column=4, sticky="ew", padx=(6, 0))
252
253 def _build_result_panel(self) -> None:
254     self.result_title = ctk.CTkLabel(
255         self.result_card,
256         text="Результат",
257         anchor="w",
258         font=self.fonts["title_small"],
259     )
260     self.result_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(18, 8))
261
262     self.status_label = ctk.CTkLabel(
263         self.result_card,
264         textvariable=self.status_var,
265         anchor="w",
266         justify="left",
267         wraplength=360,
268         font=self.fonts["result"],
269     )
270     self.status_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
271
272     self.error_label = ctk.CTkLabel(
273         self.result_card,
274         textvariable=self.error_var,
275         anchor="w",
276         justify="left",
277         wraplength=360,
278         font=self.fonts["body"],
279     )
280     self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 12))
281
282     self.stats_frame = ctk.CTkFrame(self.result_card, corner_radius=8,
283         ↪ border_width=1)
284     self.stats_frame.grid(row=3, column=0, sticky="ew", padx=20, pady=(0, 14))
285     self.stats_frame.grid_columnconfigure((0, 1), weight=1)

```

```

285
286 self.det_caption = ctk.CTkLabel(
287     self.stats_frame,
288     text="det(A)",
289     font=self.fonts["label"],
290     anchor="w",
291 )
292 self.det_caption.grid(row=0, column=0, sticky="ew", padx=14, pady=(12, 2))
293 self.det_value = ctk.CTkLabel(
294     self.stats_frame,
295     text="-",
296     font=self.fonts["metric"],
297     anchor="w",
298 )
299 self.det_value.grid(row=1, column=0, sticky="ew", padx=14, pady=(0, 12))
300
301 self.residual_caption = ctk.CTkLabel(
302     self.stats_frame,
303     text="max |Ax - b|",
304     font=self.fonts["label"],
305     anchor="w",
306 )
307 self.residual_caption.grid(row=0, column=1, sticky="ew", padx=14, pady=(12,
↵ 2))
308 self.residual_value = ctk.CTkLabel(
309     self.stats_frame,
310     text="-",
311     font=self.fonts["metric"],
312     anchor="w",
313 )
314 self.residual_value.grid(row=1, column=1, sticky="ew", padx=14, pady=(0,
↵ 12))
315
316 self.solution_label = ctk.CTkLabel(
317     self.result_card,
318     text="Вектор x",
319     anchor="w",
320     font=self.fonts["label"],
321 )
322 self.solution_label.grid(row=4, column=0, sticky="ew", padx=20, pady=(0, 6))
323
324 self.result_text = ctk.CTkTextbox(
325     self.result_card,
326     corner_radius=8,
327     border_width=1,
328     wrap="none",
329     font=self.fonts["mono"],
330 )
331 self.result_text.grid(row=5, column=0, sticky="nsew", padx=20, pady=(0, 18))
332 self._set_text(self.result_text, "Після натискання «Розв'язати» тут буде
↵ вектор x.")
333
334 def _build_matrix_grid(self, problem: MatrixProblem) -> None:
335     for child in self.matrix_area.winfo_children():

```

```

336         child.destroy()
337
338     size = problem.size
339     self.matrix_entries = []
340     self.vector_entries = []
341     self.matrix_area.grid_columnconfigure(tuple(range(size + 3)), weight=1)
342
343     empty_corner = ctk.CTkLabel(self.matrix_area, text="", width=44)
344     empty_corner.grid(row=0, column=0, padx=(0, 8), pady=(6, 8))
345
346     for column in range(size):
347         label = ctk.CTkLabel(
348             self.matrix_area,
349             text=f"x{column + 1}",
350             font=self.fonts["label"],
351             anchor="center",
352         )
353         label.grid(row=0, column=column + 1, sticky="ew", padx=4, pady=(6, 8))
354
355     separator = ctk.CTkLabel(self.matrix_area, text="|",
356         ↪ font=self.fonts["label"])
357     separator.grid(row=0, column=size + 1, padx=8, pady=(6, 8))
358
359     rhs_header = ctk.CTkLabel(
360         self.matrix_area,
361         text="b",
362         font=self.fonts["label"],
363         anchor="center",
364     )
365     rhs_header.grid(row=0, column=size + 2, sticky="ew", padx=4, pady=(6, 8))
366
367     for row in range(size):
368         row_label = ctk.CTkLabel(
369             self.matrix_area,
370             text=f"R{row + 1}",
371             width=44,
372             font=self.fonts["label"],
373             anchor="w",
374         )
375         row_label.grid(row=row + 1, column=0, sticky="w", padx=(0, 8), pady=5)
376
377         row_entries: list[ctk.CTkEntry] = []
378         for column in range(size):
379             entry = ctk.CTkEntry(
380                 self.matrix_area,
381                 width=84,
382                 height=38,
383                 justify="center",
384                 corner_radius=8,
385                 font=self.fonts["body"],
386             )
387             entry.grid(row=row + 1, column=column + 1, sticky="ew", padx=4,
388                 ↪ pady=5)
389             self._set_entry_value(entry, problem.matrix[row][column])

```

```

388         entry.bind("<KeyRelease>", self._mark_dirty)
389         row_entries.append(entry)
390
391         separator = ctk.CTkLabel(self.matrix_area, text="|",
392             ↪ font=self.fonts["label"])
393         separator.grid(row=row + 1, column=size + 1, padx=8, pady=5)
394
395         rhs_entry = ctk.CTkEntry(
396             self.matrix_area,
397             width=84,
398             height=38,
399             justify="center",
400             corner_radius=8,
401             font=self.fonts["body"],
402         )
403         rhs_entry.grid(row=row + 1, column=size + 2, sticky="ew", padx=4,
404             ↪ pady=5)
405         self._set_entry_value(rhs_entry, problem.vector[row])
406         rhs_entry.bind("<KeyRelease>", self._mark_dirty)
407         self.matrix_entries.append(row_entries)
408         self.vector_entries.append(rhs_entry)
409
410         self._style_matrix_cells(self.palette)
411
412     def _resize_matrix(self, selected: str) -> None:
413         old_matrix, old_vector = self._read_entries_loose()
414         old_size = len(old_vector)
415         new_size = int(selected)
416         matrix = [[0.0 for _ in range(new_size)] for _ in range(new_size)]
417         vector = [0.0 for _ in range(new_size)]
418
419         for row in range(min(old_size, new_size)):
420             vector[row] = old_vector[row]
421             for column in range(min(old_size, new_size)):
422                 matrix[row][column] = old_matrix[row][column]
423
424         self._set_problem(
425             MatrixProblem(
426                 name=self.name_entry.get().strip() or f"Система
427                 ↪ {new_size}x{new_size}",
428                 matrix=matrix,
429                 vector=vector,
430             ),
431             reset_output=False,
432         )
433         self.status_var.set("Розмір змінено. Заповни коефіцієнти і розв'яжи
434             ↪ систему.")
435         self.error_var.set("")
436         self.app_state.result = None
437         self.app_state.report = ""
438         self.on_result_changed()
439
440     def solve(self) -> None:
441         try:

```

```

438         problem = self._parse_problem()
439         result = solve_gauss_partial_pivot(problem)
440         report = build_full_report(result)
441
442         self.app_state.problem = problem
443         self.app_state.result = result
444         self.app_state.report = report
445
446         self.status_var.set("Систему розв'язано")
447         self.error_var.set("")
448         self.det_value.configure(text=f"{result.determinant:.6g}")
449         self.residual_value.configure(text=f"{result.residual_norm:.3e}")
450         self._set_text(self.result_text, build_short_result(result))
451         self.on_result_changed()
452     except Exception as exc:
453         self.status_var.set("Обчислення не виконано")
454         self.error_var.set(f"Помилка: {exc}")
455         self.det_value.configure(text="-")
456         self.residual_value.configure(text="-")
457         self._set_text(self.result_text, "Перевір вхідні дані та повтори
↳ обчислення.")
458
459 def load_json(self) -> None:
460     path = filedialog.askopenfilename(
461         title="Вибір JSON-файлу",
462         filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
463     )
464     if not path:
465         return
466
467     try:
468         problem = load_system_config(path)
469         validate_problem(problem)
470         self._set_problem(problem, reset_output=True)
471         self.app_state.problem = problem
472         self.app_state.result = None
473         self.app_state.report = ""
474         self.status_var.set("JSON завантажено. Натисни «Розв'язати».")
475         self.error_var.set("")
476         self.on_result_changed()
477     except Exception as exc:
478         self.status_var.set("JSON не завантажено")
479         self.error_var.set(f"Помилка: {exc}")
480
481 def save_report(self) -> None:
482     if not self.app_state.report.strip():
483         self.status_var.set("Немає звіту")
484         self.error_var.set("Спочатку виконай обчислення.")
485         return
486
487     path = filedialog.asksaveasfilename(
488         title="Збереження звіту",
489         defaultextension=".txt",
490         filetypes=[("Text files", "*.txt"), ("All files", "*.*")],

```

```

491     )
492     if not path:
493         return
494
495     try:
496         save_text_report(path, self.app_state.report)
497         self.status_var.set("Звіт збережено")
498         self.error_var.set("")
499     except Exception as exc:
500         self.status_var.set("Звіт не збережено")
501         self.error_var.set(f"Помилка: {exc}")
502
503     def reset_variant(self) -> None:
504         problem = build_default_problem()
505         self._set_problem(problem, reset_output=True)
506         self.app_state.problem = problem
507         self.app_state.result = None
508         self.app_state.report = ""
509         self.status_var.set("Варіант з фото відновлено")
510         self.error_var.set("")
511         self.on_result_changed()
512
513     def clear_matrix(self) -> None:
514         size = int(self.size_var.get())
515         problem = MatrixProblem(
516             name=f"Система {size}x{size}",
517             matrix=[[0.0 for _ in range(size)] for _ in range(size)],
518             vector=[0.0 for _ in range(size)],
519         )
520         self._set_problem(problem, reset_output=True)
521         self.app_state.problem = problem
522         self.app_state.result = None
523         self.app_state.report = ""
524         self.status_var.set("Поля очищено")
525         self.error_var.set("")
526         self.on_result_changed()
527
528     def refresh(self) -> None:
529         if self.app_state.result is not None:
530             self.status_var.set("Систему розв'язано")
531
532             ↪ self.det_value.configure(text=f"{self.app_state.result.determinant:.6g}")
533
534             ↪ self.residual_value.configure(text=f"{self.app_state.result.residual_norm:.3e}")
535             self._set_text(self.result_text,
536                 ↪ build_short_result(self.app_state.result))
537
538     def _parse_problem(self) -> MatrixProblem:
539         size = int(self.size_var.get())
540         matrix: NumberMatrix = []
541         vector: NumberVector = []
542         for row in range(size):
543             matrix_row: list[float] = []
544             for column in range(size):

```

```

542         ↪ matrix_row.append(self._parse_number(self.matrix_entries[row][column].get())
543     matrix.append(matrix_row)
544     vector.append(self._parse_number(self.vector_entries[row].get()))
545
546     problem = MatrixProblem(
547         name=self.name_entry.get().strip() or f"Система {size}x{size}",
548         matrix=matrix,
549         vector=vector,
550     )
551     validate_problem(problem)
552     return problem
553
554 def _read_entries_loose(self) -> tuple[NumberMatrix, NumberVector]:
555     size = len(self.vector_entries)
556     matrix = [[0.0 for _ in range(size)] for _ in range(size)]
557     vector = [0.0 for _ in range(size)]
558
559     for row in range(size):
560         for column in range(size):
561             try:
562                 matrix[row][column] =
563                     ↪ self._parse_number(self.matrix_entries[row][column].get())
564             except ValueError:
565                 matrix[row][column] = 0.0
566
567             try:
568                 vector[row] = self._parse_number(self.vector_entries[row].get())
569             except ValueError:
570                 vector[row] = 0.0
571
572     return matrix, vector
573
574 def _set_problem(self, problem: MatrixProblem, reset_output: bool) -> None:
575     self.size_var.set(str(problem.size))
576     self.name_entry.delete(0, tk.END)
577     self.name_entry.insert(0, problem.name)
578     self._build_matrix_grid(problem)
579     if reset_output:
580         self.det_value.configure(text="-")
581         self.residual_value.configure(text="-")
582         self._set_text(self.result_text, "Після натискання «Розв'язати» тут буде
583             ↪ вектор x.")
584
585 def _parse_number(self, raw_value: str) -> float:
586     normalized = raw_value.strip().replace(",", ".")
587     if not normalized:
588         raise ValueError("Порожні поля потрібно заповнити числами.")
589     return float(normalized)
590
591 def _set_entry_value(self, entry: ctk.CTkEntry, value: object) -> None:
592     entry.delete(0, tk.END)
593     if isinstance(value, float):
594         entry.insert(0, f"{value:g}")
595     else:

```

```

593         entry.insert(0, str(value))
594
595     def _set_text(self, textbox: ctk.CTkTextbox, text: str) -> None:
596         textbox.configure(state="normal")
597         textbox.delete("1.0", tk.END)
598         textbox.insert("1.0", text)
599         textbox.configure(state="disabled")
600
601     def _mark_dirty(self, _event: tk.Event | None = None) -> None:
602         if self.app_state.result is None and not self.app_state.report:
603             return
604
605         self.app_state.result = None
606         self.app_state.report = ""
607         self.status_var.set("Дані змінено")
608         self.error_var.set("Результат застарів. Запусти обчислення ще раз.")
609         self.det_value.configure(text="-")
610         self.residual_value.configure(text="-")
611         self._set_text(self.result_text, "Матрицю змінено. Натисни «Розв'язати», щоб
        ↪ оновити результат.")
612         self.on_result_changed()
613
614     def _style_matrix_cells(self, palette: dict[str, str]) -> None:
615         for row in self.matrix_entries:
616             for entry in row:
617                 entry.configure(
618                     fg_color=palette["panel_alt"],
619                     border_color=palette["surface_soft"],
620                     text_color=palette["text"],
621                 )
622         for entry in self.vector_entries:
623             entry.configure(
624                 fg_color=palette["panel_alt"],
625                 border_color=palette["warning"],
626                 text_color=palette["text"],
627             )
628
629     def apply_theme(self, palette: dict[str, str]) -> None:
630         super().apply_theme(palette)
631         for card in (self.header_card, self.input_card, self.result_card,
        ↪ self.stats_frame):
632             card.configure(fg_color=palette["panel"],
        ↪ border_color=palette["surface_soft"])
633
634         self.matrix_area.configure(
635             fg_color=palette["surface"],
636             border_color=palette["surface_soft"],
637             scrollbar_button_color=palette["segment"],
638             scrollbar_button_hover_color=palette["segment_hover"],
639         )
640
641         for label in (
642             self.title_label,
643             self.result_title,

```

```

644         self.det_value,
645         self.residual_value,
646     ):
647         label.configure(text_color=palette["text"])
648
649     for label in (
650         self.subtitle_label,
651         self.name_label,
652         self.size_label,
653         self.method_label,
654         self.det_caption,
655         self.residual_caption,
656         self.solution_label,
657     ):
658         label.configure(text_color=palette["muted"])
659
660     self.status_label.configure(text_color=palette["text"])
661     self.error_label.configure(text_color=palette["error"])
662     self.name_entry.configure(
663         fg_color=palette["surface"],
664         border_color=palette["surface_soft"],
665         text_color=palette["text"],
666     )
667     self.size_menu.configure(
668         fg_color=palette["segment"],
669         button_color=palette["segment"],
670         button_hover_color=palette["segment_hover"],
671         text_color=palette["text"],
672     )
673     self.result_text.configure(
674         fg_color=palette["surface"],
675         border_color=palette["surface_soft"],
676         text_color=palette["text"],
677     )
678     self._style_matrix_cells(palette)
679
680     self.solve_button.configure(
681         fg_color=palette["accent"],
682         hover_color=palette["accent_hover"],
683         text_color=palette["accent_text"],
684     )
685     for button in (
686         self.load_button,
687         self.save_button,
688         self.default_button,
689         self.clear_button,
690     ):
691         button.configure(
692             fg_color=palette["segment"],
693             hover_color=palette["segment_hover"],
694             text_color=palette["text"],
695         )
696
697

```

```

698 class StepsPage(BasePage):
699     def __init__(
700         self,
701         master: ctk.CTkFrame,
702         page_config: PageConfig,
703         fonts: dict[str, ctk.CTkFont],
704         app_state: LinearSystemState,
705     ) -> None:
706         super().__init__(master, page_config, fonts)
707         self.app_state = app_state
708         self.steps: list[EliminationStep] = []
709
710         self.grid_rowconfigure(1, weight=1)
711         self._build_header()
712         self._build_body()
713
714     def _build_header(self) -> None:
715         self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
716         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
717         self.header_card.grid_columnconfigure(0, weight=1)
718
719         self.title_label = ctk.CTkLabel(
720             self.header_card,
721             text=self.page_config.title,
722             anchor="w",
723             font=self.fonts["title"],
724         )
725         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
726
727         self.subtitle_label = ctk.CTkLabel(
728             self.header_card,
729             text=self.page_config.subtitle,
730             anchor="w",
731             justify="left",
732             wraplength=1020,
733             font=self.fonts["subtitle"],
734         )
735         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
736             ↪ 14))
737
738     def _build_body(self) -> None:
739         self.body = ctk.CTkFrame(self, fg_color="transparent")
740         self.body.grid(row=1, column=0, sticky="nsew")
741         self.body.grid_columnconfigure(0, weight=3)
742         self.body.grid_columnconfigure(1, weight=4)
743         self.body.grid_rowconfigure(0, weight=1)
744
745         self.table_card = ctk.CTkFrame(self.body, corner_radius=8, border_width=1)
746         self.table_card.grid(row=0, column=0, sticky="nsew", padx=(0, 10))
747         self.table_card.grid_columnconfigure(0, weight=1)
748         self.table_card.grid_rowconfigure(1, weight=1)
749
750         self.table_title = ctk.CTkLabel(
751             self.table_card,

```

```

751         text="Вибір головних елементів",
752         anchor="w",
753         font=self.fonts["title_small"],
754     )
755     self.table_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18, 8))
756
757     self.step_table_frame = tk.Frame(self.table_card, bg=MAIN_THEME["panel"])
758     self.step_table_frame.grid(row=1, column=0, sticky="nsew", padx=18, pady=(0,
759         ↪ 18))
760     self.step_table_frame.grid_columnconfigure(0, weight=1)
761     self.step_table_frame.grid_rowconfigure(0, weight=1)
762
763     columns = ("step", "column", "row", "pivot", "swap", "ops")
764     self.step_table = ttk.Treeview(
765         self.step_table_frame,
766         columns=columns,
767         show="headings",
768         height=14,
769     )
770     headings = {
771         "step": "k",
772         "column": "Стовпець",
773         "row": "Рядок",
774         "pivot": "Pivot",
775         "swap": "Обмін",
776         "ops": "Опер.",
777     }
778     widths = {
779         "step": 44,
780         "column": 82,
781         "row": 72,
782         "pivot": 112,
783         "swap": 74,
784         "ops": 62,
785     }
786     for column in columns:
787         self.step_table.heading(column, text=headings[column])
788         self.step_table.column(column, width=widths[column], anchor="center")
789
790     self.step_scrollbar = ttk.Scrollbar(
791         self.step_table_frame,
792         orient="vertical",
793         command=self.step_table.yview,
794     )
795     self.step_table.configure(yscrollcommand=self.step_scrollbar.set)
796     self.step_table.grid(row=0, column=0, sticky="nsew")
797     self.step_scrollbar.grid(row=0, column=1, sticky="ns")
798     self.step_table.bind("<<TreeviewSelect>>", self._select_step)
799
800     self.details_card = ctk.CTkFrame(self.body, corner_radius=8, border_width=1)
801     self.details_card.grid(row=0, column=1, sticky="nsew", padx=(10, 0))
802     self.details_card.grid_columnconfigure(0, weight=1)
803     self.details_card.grid_rowconfigure(1, weight=1)

```

```

804     self.details_title = ctk.CTkLabel(
805         self.details_card,
806         text="Матриця після кроку",
807         anchor="w",
808         font=self.fonts["title_small"],
809     )
810     self.details_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18, 8))
811
812     self.details_text = ctk.CTkTextbox(
813         self.details_card,
814         corner_radius=8,
815         border_width=1,
816         wrap="none",
817         font=self.fonts["mono"],
818     )
819     self.details_text.grid(row=1, column=0, sticky="nsew", padx=18, pady=(0,
820 ↵ 18))
821     self._set_text("Спочатку розв'яжи систему на сторінці «Система».")
822
823 def refresh(self) -> None:
824     for item in self.step_table.get_children():
825         self.step_table.delete(item)
826
827     result = self.app_state.result
828     self.steps = [] if result is None else result.elimination_steps
829     if result is None:
830         self._set_text("Спочатку розв'яжи систему на сторінці «Система».")
831         return
832
833     for index, step in enumerate(self.steps):
834         self.step_table.insert(
835             "",
836             "end",
837             iid=str(index),
838             values=(
839                 step.step,
840                 step.pivot_column + 1,
841                 step.selected_row + 1,
842                 f"{step.pivot_value:.6g}",
843                 "так" if step.did_swap else "ні",
844                 len(step.operations),
845             ),
846         )
847
848     if self.steps:
849         first_item = self.step_table.get_children()[0]
850         self.step_table.selection_set(first_item)
851         self.step_table.focus(first_item)
852         self._show_step(self.steps[0])
853     else:
854         self._set_text("Для системи 1x1 прямий хід не потрібен.")
855
856 def _select_step(self, _event: tk.Event) -> None:
857     selected = self.step_table.selection()

```

```

857         if not selected:
858             return
859         step_index = int(selected[0])
860         if 0 <= step_index < len(self.steps):
861             self._show_step(self.steps[step_index])
862
863     def _show_step(self, step: EliminationStep) -> None:
864         lines = [
865             f"Крок {step.step}",
866             f"Стовпець: {step.pivot_column + 1}",
867             f"Вибрано рядок: R{step.selected_row + 1}",
868             f"Головний елемент: {step.pivot_value:.12g}",
869         ]
870         if step.did_swap:
871             lines.append(f"Перестановка: R{step.pivot_row + 1} <->
872                 ↪ R{step.selected_row + 1}")
873         else:
874             lines.append("Перестановка: не потрібна")
875
876         lines.append("")
877         lines.append("Операції занулення:")
878         if step.operations:
879             lines.extend(operation.as_text() for operation in step.operations)
880         else:
881             lines.append("Немає")
882
883         lines.append("")
884         lines.append(format_matrix_for_display(step.matrix, step.vector))
885         self._set_text("\n".join(lines))
886
887     def _set_text(self, text: str) -> None:
888         self.details_text.configure(state="normal")
889         self.details_text.delete("1.0", tk.END)
890         self.details_text.insert("1.0", text)
891         self.details_text.configure(state="disabled")
892
893     def apply_theme(self, palette: dict[str, str]) -> None:
894         super().apply_theme(palette)
895         for card in (self.header_card, self.table_card, self.details_card):
896             card.configure(fg_color=palette["panel"],
897                 ↪ border_color=palette["surface_soft"])
898
899         self.title_label.configure(text_color=palette["text"])
900         self.subtitle_label.configure(text_color=palette["muted"])
901         self.table_title.configure(text_color=palette["text"])
902         self.details_title.configure(text_color=palette["text"])
903         self.step_table_frame.configure(bg=palette["panel"])
904         self.details_text.configure(
905             fg_color=palette["surface"],
906             border_color=palette["surface_soft"],
907             text_color=palette["text"],
908         )

```

```

909 class CheckPage(BasePage):
910     def __init__(
911         self,
912         master: ctk.CTkFrame,
913         page_config: PageConfig,
914         fonts: dict[str, ctk.CTkFont],
915         app_state: LinearSystemState,
916     ) -> None:
917         super().__init__(master, page_config, fonts)
918         self.app_state = app_state
919
920         self.grid_rowconfigure(1, weight=1)
921         self._build_header()
922         self._build_body()
923
924     def _build_header(self) -> None:
925         self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
926         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
927         self.header_card.grid_columnconfigure(0, weight=1)
928
929         self.title_label = ctk.CTkLabel(
930             self.header_card,
931             text=self.page_config.title,
932             anchor="w",
933             font=self.fonts["title"],
934         )
935         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
936
937         self.subtitle_label = ctk.CTkLabel(
938             self.header_card,
939             text=self.page_config.subtitle,
940             anchor="w",
941             justify="left",
942             wraplength=1020,
943             font=self.fonts["subtitle"],
944         )
945         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
946             ↪ 14))
947
948     def _build_body(self) -> None:
949         self.body = ctk.CTkFrame(self, fg_color="transparent")
950         self.body.grid(row=1, column=0, sticky="nsew")
951         self.body.grid_columnconfigure((0, 1), weight=1)
952         self.body.grid_rowconfigure(1, weight=1)
953
954         self.solution_card = ctk.CTkFrame(self.body, corner_radius=8,
955             ↪ border_width=1)
956         self.solution_card.grid(row=0, column=0, sticky="nsew", padx=(0, 10),
957             ↪ pady=(0, 14))
958         self.solution_card.grid_columnconfigure(0, weight=1)
959         self.solution_card.grid_rowconfigure(1, weight=1)
960
961         self.residual_card = ctk.CTkFrame(self.body, corner_radius=8,
962             ↪ border_width=1)

```

```

959 self.residual_card.grid(row=0, column=1, sticky="nsew", padx=(10, 0),
    ↪ pady=(0, 14))
960 self.residual_card.grid_columnconfigure(0, weight=1)
961 self.residual_card.grid_rowconfigure(1, weight=1)
962
963 self.chart_card = ctk.CTkFrame(self.body, corner_radius=8, border_width=1)
964 self.chart_card.grid(row=1, column=0, columnspan=2, sticky="nsew")
965 self.chart_card.grid_columnconfigure(0, weight=1)
966 self.chart_card.grid_rowconfigure(1, weight=1)
967
968 self.solution_title = ctk.CTkLabel(
969     self.solution_card,
970     text="Вектор розв'язку",
971     anchor="w",
972     font=self.fonts["title_small"],
973 )
974 self.solution_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18,
    ↪ 8))
975
976 self.solution_table_frame = tk.Frame(self.solution_card,
    ↪ bg=MAIN_THEME["panel"])
977 self.solution_table_frame.grid(row=1, column=0, sticky="nsew", padx=18,
    ↪ pady=(0, 18))
978 self.solution_table_frame.grid_columnconfigure(0, weight=1)
979 self.solution_table_frame.grid_rowconfigure(0, weight=1)
980
981 self.solution_table = ttk.Treeview(
982     self.solution_table_frame,
983     columns=("variable", "value", "rounded"),
984     show="headings",
985     height=7,
986 )
987 for column, title, width in (
988     ("variable", "Змінна", 80),
989     ("value", "Значення", 180),
990     ("rounded", "До 4 знаків", 140),
991 ):
992     self.solution_table.heading(column, text=title)
993     self.solution_table.column(column, width=width, anchor="center")
994 self.solution_table.grid(row=0, column=0, sticky="nsew")
995
996 self.residual_title = ctk.CTkLabel(
997     self.residual_card,
998     text="Нев'язки",
999     anchor="w",
1000     font=self.fonts["title_small"],
1001 )
1002 self.residual_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18,
    ↪ 8))
1003
1004 self.residual_table_frame = tk.Frame(self.residual_card,
    ↪ bg=MAIN_THEME["panel"])
1005 self.residual_table_frame.grid(row=1, column=0, sticky="nsew", padx=18,
    ↪ pady=(0, 18))

```

```

1006 self.residual_table_frame.grid_columnconfigure(0, weight=1)
1007 self.residual_table_frame.grid_rowconfigure(0, weight=1)
1008
1009 self.residual_table = ttk.Treeview(
1010     self.residual_table_frame,
1011     columns=("row", "residual", "absolute"),
1012     show="headings",
1013     height=7,
1014 )
1015 for column, title, width in (
1016     ("row", "Рядок", 80),
1017     ("residual", "Ax - b", 180),
1018     ("absolute", "|r|", 140),
1019 ):
1020     self.residual_table.heading(column, text=title)
1021     self.residual_table.column(column, width=width, anchor="center")
1022 self.residual_table.grid(row=0, column=0, sticky="nsew")
1023
1024 self.chart_title = ctk.CTkLabel(
1025     self.chart_card,
1026     text="Графік модулів нев'язки",
1027     anchor="w",
1028     font=self.fonts["title_small"],
1029 )
1030 self.chart_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18, 8))
1031
1032 self.chart_canvas = tk.Canvas(
1033     self.chart_card,
1034     height=280,
1035     bg=MAIN_THEME["surface"],
1036     highlightthickness=0,
1037 )
1038 self.chart_canvas.grid(row=1, column=0, sticky="nsew", padx=18, pady=(0,
1039     ↪ 18))
1040
1041 def refresh(self) -> None:
1042     for table in (self.solution_table, self.residual_table):
1043         for item in table.get_children():
1044             table.delete(item)
1045
1046     result = self.app_state.result
1047     if result is None:
1048         self._draw_empty_chart("Спочатку розв'яжи систему на сторінці
1049             ↪ «Система».")
1050         return
1051
1052     for index, value in enumerate(result.solution, start=1):
1053         self.solution_table.insert(
1054             "",
1055             "end",
1056             values=(f"x{index}", f"{value:.10f}", f"{value:.4f}"),
1057         )
1058
1059     for index, value in enumerate(result.residuals, start=1):

```

```

1058         self.residual_table.insert(
1059             "",
1060             "end",
1061             values=(f"R{index}", f"{value:.3e}", f"{abs(value):.3e}"),
1062         )
1063
1064     self._draw_residual_chart(result.residuals)
1065
1066 def _draw_empty_chart(self, text: str) -> None:
1067     palette = self.palette
1068     self.chart_canvas.delete("all")
1069     width = max(self.chart_canvas.winfo_width(), 720)
1070     height = max(self.chart_canvas.winfo_height(), 260)
1071     self.chart_canvas.create_rectangle(0, 0, width, height,
1072         ↪ fill=palette["surface"], outline="")
1073     self.chart_canvas.create_text(
1074         width / 2,
1075         height / 2,
1076         text=text,
1077         fill=palette["muted"],
1078         font=("Segoe UI", 13, "bold"),
1079     )
1080
1081 def _draw_residual_chart(self, residuals: NumberVector) -> None:
1082     palette = self.palette
1083     self.chart_canvas.delete("all")
1084     width = max(self.chart_canvas.winfo_width(), 720)
1085     height = max(self.chart_canvas.winfo_height(), 260)
1086     left = 84
1087     right = 34
1088     top = 32
1089     bottom = 42
1090     plot_width = width - left - right
1091     plot_height = height - top - bottom
1092
1093     self.chart_canvas.create_rectangle(0, 0, width, height,
1094         ↪ fill=palette["surface"], outline="")
1095     self.chart_canvas.create_rectangle(
1096         left,
1097         top,
1098         width - right,
1099         height - bottom,
1100         fill=palette["panel_alt"],
1101         outline=palette["surface_soft"],
1102     )
1103
1104     magnitudes = [abs(value) for value in residuals]
1105     max_value = max(magnitudes + [1e-16])
1106     bar_count = len(magnitudes)
1107     if bar_count == 0:
1108         return
1109
1110     slot_width = plot_width / bar_count
1111     for index, magnitude in enumerate(magnitudes):

```

```

1110         x_center = left + slot_width * index + slot_width / 2
1111         bar_width = min(72, slot_width * 0.48)
1112         normalized_height = (magnitude / max_value) * (plot_height - 24)
1113         y_top = height - bottom - normalized_height
1114
1115         fill = palette["accent"] if magnitude <= 1e-8 else palette["warning"]
1116         if magnitude > 1e-5:
1117             fill = palette["error"]
1118
1119         self.chart_canvas.create_rectangle(
1120             x_center - bar_width / 2,
1121             y_top,
1122             x_center + bar_width / 2,
1123             height - bottom,
1124             fill=fill,
1125             outline="",
1126         )
1127         self.chart_canvas.create_text(
1128             x_center,
1129             height - 20,
1130             text=f"r{index + 1}",
1131             fill=palette["muted"],
1132             font=("Segoe UI", 10, "bold"),
1133         )
1134         self.chart_canvas.create_text(
1135             x_center,
1136             max(top + 14, y_top - 12),
1137             text=f"{magnitude:.1e}",
1138             fill=palette["text"],
1139             font=("Segoe UI", 9),
1140         )
1141
1142         self.chart_canvas.create_text(
1143             left,
1144             16,
1145             text=f"max |r| = {max_value:.3e}",
1146             fill=palette["text"],
1147             anchor="w",
1148             font=("Segoe UI", 11, "bold"),
1149         )
1150
1151     def apply_theme(self, palette: dict[str, str]) -> None:
1152         super().apply_theme(palette)
1153         for card in (self.header_card, self.solution_card, self.residual_card,
1154                     ↪ self.chart_card):
1155             card.configure(fg_color=palette["panel"],
1156                           ↪ border_color=palette["surface_soft"])
1157
1158         self.title_label.configure(text_color=palette["text"])
1159         self.subtitle_label.configure(text_color=palette["muted"])
1160         self.solution_title.configure(text_color=palette["text"])
1161         self.residual_title.configure(text_color=palette["text"])
1162         self.chart_title.configure(text_color=palette["text"])
1163         self.solution_table_frame.configure(bg=palette["panel"])

```

```

1162         self.residual_table_frame.configure(bg=palette["panel"])
1163         self.chart_canvas.configure(bg=palette["surface"])
1164
1165
1166 class ExtraPage(BasePage):
1167     def __init__(
1168         self,
1169         master: ctk.CTkFrame,
1170         page_config: PageConfig,
1171         fonts: dict[str, ctk.CTkFont],
1172         app_state: LinearSystemState,
1173         on_theme_toggle: Callable[[], str],
1174     ) -> None:
1175         super().__init__(master, page_config, fonts)
1176         self.app_state = app_state
1177         self.on_theme_toggle = on_theme_toggle
1178         self.status_var = tk.StringVar(value="Готово до додаткових дій")
1179         self.effect_particles: list[EffectParticle] = []
1180         self.effect_after_id: str | None = None
1181         self.portrait_after_id: str | None = None
1182         self.gauss_image_refs: list[ImageTk.PhotoImage] = []
1183         self.music_process: subprocess.Popen[bytes] | None = None
1184
1185         self.grid_rowconfigure(1, weight=1)
1186         self._build_header()
1187         self._build_body()
1188
1189     def _build_header(self) -> None:
1190         self.header_card = ctk.CTkFrame(self, corner_radius=8, border_width=1)
1191         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
1192         self.header_card.grid_columnconfigure(0, weight=1)
1193
1194         self.title_label = ctk.CTkLabel(
1195             self.header_card,
1196             text=self.page_config.title,
1197             anchor="w",
1198             font=self.fonts["title"],
1199         )
1200         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
1201
1202         self.subtitle_label = ctk.CTkLabel(
1203             self.header_card,
1204             text=self.page_config.subtitle,
1205             anchor="w",
1206             justify="left",
1207             wraplength=1020,
1208             font=self.fonts["subtitle"],
1209         )
1210         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
1211             ↵ 14))
1212
1213     def _build_body(self) -> None:
1214         self.body = ctk.CTkFrame(self, fg_color="transparent")
1215         self.body.grid(row=1, column=0, sticky="nsew")

```

```

1215 self.body.grid_columnconfigure(0, weight=2)
1216 self.body.grid_columnconfigure(1, weight=5)
1217 self.body.grid_rowconfigure(0, weight=1)
1218
1219 self.controls_card = ctk.CTkFrame(self.body, corner_radius=8,
    ↪ border_width=1)
1220 self.controls_card.grid(row=0, column=0, sticky="nsew", padx=(0, 10))
1221 self.controls_card.grid_columnconfigure(0, weight=1)
1222
1223 self.art_card = ctk.CTkFrame(self.body, corner_radius=8, border_width=1)
1224 self.art_card.grid(row=0, column=1, sticky="nsew", padx=(10, 0))
1225 self.art_card.grid_columnconfigure(0, weight=1)
1226 self.art_card.grid_rowconfigure(1, weight=1)
1227
1228 self.controls_title = ctk.CTkLabel(
1229     self.controls_card,
1230     text="Панель ефектів",
1231     anchor="w",
1232     font=self.fonts["title_small"],
1233 )
1234 self.controls_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18,
    ↪ 8))
1235
1236 self.status_label = ctk.CTkLabel(
1237     self.controls_card,
1238     textvariable=self.status_var,
1239     anchor="w",
1240     justify="left",
1241     wraplength=260,
1242     font=self.fonts["body"],
1243 )
1244 self.status_label.grid(row=1, column=0, sticky="ew", padx=18, pady=(0, 14))
1245
1246 self.effect_button = ctk.CTkButton(
1247     self.controls_card,
1248     text="Запустити ефект",
1249     height=42,
1250     corner_radius=8,
1251     font=self.fonts["button"],
1252     command=self.play_effect,
1253 )
1254 self.effect_button.grid(row=2, column=0, sticky="ew", padx=18, pady=(0, 10))
1255
1256 self.theme_button = ctk.CTkButton(
1257     self.controls_card,
1258     text="Рожевий режим",
1259     height=42,
1260     corner_radius=8,
1261     font=self.fonts["button"],
1262     command=self.toggle_theme,
1263 )
1264 self.theme_button.grid(row=3, column=0, sticky="ew", padx=18, pady=(0, 10))
1265
1266 self.gauss_button = ctk.CTkButton(

```

```

1267         self.controls_card,
1268         text="Намалювати Гауса",
1269         height=42,
1270         corner_radius=8,
1271         font=self.fonts["button"],
1272         command=self.draw_gauss_portrait,
1273     )
1274     self.gauss_button.grid(row=4, column=0, sticky="ew", padx=18, pady=(0, 10))
1275
1276     self.music_button = ctk.CTkButton(
1277         self.controls_card,
1278         text="Грати музику",
1279         height=42,
1280         corner_radius=8,
1281         font=self.fonts["button"],
1282         command=self.toggle_music,
1283     )
1284     self.music_button.grid(row=5, column=0, sticky="ew", padx=18, pady=(0, 10))
1285
1286     self.clear_button = ctk.CTkButton(
1287         self.controls_card,
1288         text="Очистити полотно",
1289         height=42,
1290         corner_radius=8,
1291         font=self.fonts["button"],
1292         command=self.clear_canvas,
1293     )
1294     self.clear_button.grid(row=6, column=0, sticky="ew", padx=18, pady=(0, 18))
1295
1296     self.note_label = ctk.CTkLabel(
1297         self.controls_card,
1298         text=(
1299             "Портрет Гауса береться з локального reference-зображення, "
1300             "а полотно розкриває його з анімованим блиском."
1301         ),
1302         anchor="w",
1303         justify="left",
1304         wraplength=260,
1305         font=self.fonts["subtitle"],
1306     )
1307     self.note_label.grid(row=7, column=0, sticky="ew", padx=18, pady=(0, 18))
1308
1309     self.art_title = ctk.CTkLabel(
1310         self.art_card,
1311         text="Полотно",
1312         anchor="w",
1313         font=self.fonts["title_small"],
1314     )
1315     self.art_title.grid(row=0, column=0, sticky="ew", padx=18, pady=(18, 8))
1316
1317     self.canvas = tk.Canvas(
1318         self.art_card,
1319         bg=MAIN_THEME["surface"],
1320         highlightthickness=0,

```

```

1321     )
1322     self.canvas.grid(row=1, column=0, sticky="nsew", padx=18, pady=(0, 18))
1323     self.canvas.bind("<Configure>", self._redraw_idle_if_empty)
1324     self.clear_canvas()
1325
1326     def refresh(self) -> None:
1327         if not self.canvas.find_all():
1328             self.clear_canvas()
1329
1330     def toggle_theme(self) -> None:
1331         message = self.on_theme_toggle()
1332         self.status_var.set(message)
1333         if not self.gauss_image_refs and not self.effect_particles:
1334             self.clear_canvas()
1335
1336     def toggle_music(self) -> None:
1337         if self.music_process is not None and self.music_process.poll() is None:
1338             self._stop_music()
1339             self.music_button.configure(text="Грати музику")
1340             self.status_var.set("Музику зупинено")
1341             return
1342
1343         try:
1344             music_path = self._find_music_path()
1345             self._start_music_process(music_path)
1346             self.music_button.configure(text="Зупинити музику")
1347             self.status_var.set(f"Грає: {music_path.name}")
1348         except Exception as exc:
1349             self._stop_music()
1350             self.music_button.configure(text="Грати музику")
1351             self.status_var.set(f"Музику не запущено: {exc}")
1352
1353     def _find_music_path(self) -> Path:
1354         for path in MUSIC_CANDIDATES:
1355             if path.exists():
1356                 return path
1357         raise FileNotFoundError("не знайдено assets.mp3 або MP3 у project/assets")
1358
1359     def _stop_music(self) -> None:
1360         if self.music_process is not None and self.music_process.poll() is None:
1361             self.music_process.terminate()
1362             try:
1363                 self.music_process.wait(timeout=2)
1364             except subprocess.TimeoutExpired:
1365                 self.music_process.kill()
1366         self.music_process = None
1367
1368     def _start_music_process(self, music_path: Path) -> None:
1369         self._stop_music()
1370         script = f"""
1371 Add-Type -AssemblyName presentationCore
1372 $player = New-Object System.Windows.Media.MediaPlayer
1373 $player.Open([Uri] '{music_path}')
1374 $player.Volume = 0.85

```

```

1375 $player.Play()
1376 while ($true) {{
1377     Start-Sleep -Milliseconds 400
1378     if ($player.NaturalDuration.HasTimeSpan -and $player.Position -ge
1379         ↪ $player.NaturalDuration.TimeSpan) {{
1380         $player.Position = [TimeSpan]::Zero
1381         $player.Play()
1382     }}
1383 }}
1384 """
1385     encoded = base64.b64encode(script.encode("utf-16le")).decode("ascii")
1386     creationflags = getattr(subprocess, "CREATE_NO_WINDOW", 0)
1387     self.music_process = subprocess.Popen(
1388         [
1389             "powershell",
1390             "-NoProfile",
1391             "-ExecutionPolicy",
1392             "Bypass",
1393             "-EncodedCommand",
1394             encoded,
1395         ],
1396         stdout=subprocess.DEVNULL,
1397         stderr=subprocess.DEVNULL,
1398         creationflags=creationflags,
1399     )
1400 def play_effect(self) -> None:
1401     if self.effect_after_id is not None:
1402         self.after_cancel(self.effect_after_id)
1403         self.effect_after_id = None
1404
1405     width, height = self._canvas_size()
1406     palette = self.palette
1407     colors = [
1408         palette["accent"],
1409         palette["secondary"],
1410         palette["warning"],
1411         palette["error"],
1412         palette["text"],
1413     ]
1414     symbols = ["A", "x", "b", "det", "Σ", "0", "=", "♡", "*"]
1415     self.effect_particles = [
1416         EffectParticle(
1417             x=random.uniform(24, width - 24),
1418             y=random.uniform(-height * 0.5, 32),
1419             dx=random.uniform(-1.8, 1.8),
1420             dy=random.uniform(1.8, 4.8),
1421             radius=random.uniform(3.0, 8.0),
1422             color=random.choice(colors),
1423             symbol=random.choice(symbols),
1424         )
1425         for _ in range(94)
1426     ]
1427     self.status_var.set("Ефект запущено")

```

```

1428         self._animate_effect(0)
1429
1430     def _animate_effect(self, frame: int) -> None:
1431         palette = self.palette
1432         width, height = self._canvas_size()
1433         self._draw_canvas_background()
1434
1435         glow = max(0, 18 - frame)
1436         if glow:
1437             self.canvas.create_rectangle(
1438                 18 + glow,
1439                 18 + glow,
1440                 width - 18 - glow,
1441                 height - 18 - glow,
1442                 outline=palette["accent"],
1443                 width=2,
1444             )
1445
1446         for particle in self.effect_particles:
1447             particle.x += particle.dx
1448             particle.y += particle.dy
1449             particle.dy += 0.09
1450             if particle.x < 10 or particle.x > width - 10:
1451                 particle.dx *= -0.8
1452
1453             self.canvas.create_oval(
1454                 particle.x - particle.radius,
1455                 particle.y - particle.radius,
1456                 particle.x + particle.radius,
1457                 particle.y + particle.radius,
1458                 fill=particle.color,
1459                 outline="",
1460             )
1461             self.canvas.create_text(
1462                 particle.x,
1463                 particle.y,
1464                 text=particle.symbol,
1465                 fill=palette["surface"],
1466                 font=("Segoe UI", 8, "bold"),
1467             )
1468
1469         message = "Pivot mode"
1470         if self.app_state.result is not None:
1471             message = f"max |r| = {self.app_state.result.residual_norm:.1e}"
1472         self.canvas.create_text(
1473             width / 2,
1474             height / 2,
1475             text=message,
1476             fill=palette["text"],
1477             font=("Segoe UI", 24, "bold"),
1478         )
1479
1480         if frame >= 54:
1481             self.effect_after_id = None

```

```

1482         self.effect_particles = []
1483         self.status_var.set("Эффект завершено")
1484         return
1485
1486     self.effect_after_id = self.after(22, lambda: self._animate_effect(frame +
1487         ↪ 1))
1488
1489 def draw_gauss_portrait(self) -> None:
1490     if self.effect_after_id is not None:
1491         self.after_cancel(self.effect_after_id)
1492         self.effect_after_id = None
1493     if self.portrait_after_id is not None:
1494         self.after_cancel(self.portrait_after_id)
1495         self.portrait_after_id = None
1496     self.effect_particles = []
1497     self.gauss_image_refs = []
1498
1499     try:
1500         image, left, top = self._load_resized_gauss_image()
1501     except Exception as exc:
1502         self.clear_canvas()
1503         self.status_var.set(f"Портрет не завантажено: {exc}")
1504         return
1505
1506     self._draw_canvas_background()
1507     width, _height = self._canvas_size()
1508     self.canvas.create_text(
1509         width / 2,
1510         42,
1511         text="Carl Friedrich Gauss",
1512         fill=self.palette["text"],
1513         font=("Segoe UI", 24, "bold"),
1514     )
1515     self.canvas.create_text(
1516         width / 2,
1517         72,
1518         text="reference portrait",
1519         fill=self.palette["muted"],
1520         font=("Segoe UI", 12, "bold"),
1521     )
1522     self.status_var.set("Портрет Гауса малюється")
1523     self._paint_gauss_slice(image, left, top, 0, 22)
1524
1525 def _load_resized_gauss_image(self) -> tuple[Image.Image, int, int]:
1526     if not GAUSS_REFERENCE_PATH.exists():
1527         raise FileNotFoundError("Немає project/assets/gauss_reference.jpeg")
1528
1529     source = Image.open(GAUSS_REFERENCE_PATH).convert("RGB")
1530     width, height = self._canvas_size()
1531     max_width = int(width * 0.86)
1532     max_height = int(height * 0.66)
1533     scale = min(max_width / source.width, max_height / source.height)
1534     new_size = (
1535         max(1, int(source.width * scale)),

```

```

1535         max(1, int(source.height * scale)),
1536     )
1537     image = source.resize(new_size, Image.Resampling.LANCZOS)
1538     left = int((width - image.width) / 2)
1539     top = 94
1540     return image, left, top
1541
1542 def _paint_gauss_slice(
1543     self,
1544     image: Image.Image,
1545     left: int,
1546     top: int,
1547     index: int,
1548     slices: int,
1549 ) -> None:
1550     stripe_width = max(1, image.width // slices)
1551     stripe_left = index * stripe_width
1552     stripe_right = image.width if index == slices - 1 else min(image.width,
1553         ↪ stripe_left + stripe_width)
1554     stripe = image.crop((stripe_left, 0, stripe_right, image.height))
1555     photo = ImageTk.PhotoImage(stripe)
1556     self.gauss_image_refs.append(photo)
1557     self.canvas.create_image(left + stripe_left, top, image=photo, anchor="nw")
1558
1559     sparkle_x = left + stripe_right
1560     for offset in (-26, 0, 26):
1561         self.canvas.create_text(
1562             sparkle_x,
1563             top + image.height / 2 + offset,
1564             text="*",
1565             fill=self.palette["accent"],
1566             font=("Segoe UI", 16, "bold"),
1567         )
1568
1569     if index >= slices - 1:
1570         self.canvas.create_rectangle(
1571             left,
1572             top,
1573             left + image.width,
1574             top + image.height,
1575             outline=self.palette["surface_strong"],
1576             width=3,
1577         )
1578         width, height = self._canvas_size()
1579         self._draw_gauss_curve(width, height)
1580         self.canvas.create_text(
1581             width - 36,
1582             height - 34,
1583             text="e^{-x^2}",
1584             fill=self.palette["secondary"],
1585             anchor="e",
1586             font=("Consolas", 15, "bold"),
1587         )
1588     self.status_var.set("Портрет Гауса намальовано")

```

```

1588         self.portrait_after_id = None
1589         return
1590
1591     self.portrait_after_id = self.after(
1592         36,
1593         lambda: self._paint_gauss_slice(image, left, top, index + 1, slices),
1594     )
1595
1596     def clear_canvas(self) -> None:
1597         if self.effect_after_id is not None:
1598             self.after_cancel(self.effect_after_id)
1599             self.effect_after_id = None
1600         if self.portrait_after_id is not None:
1601             self.after_cancel(self.portrait_after_id)
1602             self.portrait_after_id = None
1603         self.effect_particles = []
1604         self.gauss_image_refs = []
1605         self._draw_canvas_background()
1606         width, height = self._canvas_size()
1607         self.canvas.create_text(
1608             width / 2,
1609             height / 2,
1610             text="Тут з'явиться ефект або портрет Гауса",
1611             fill=self.palette["subtle"],
1612             font=("Segoe UI", 15, "bold"),
1613         )
1614         self.status_var.set("Полотно очищено")
1615
1616     def _draw_gauss_curve(self, width: int, height: int) -> None:
1617         palette = self.palette
1618         base_y = height - 58
1619         start_x = 58
1620         end_x = width - 58
1621         self.canvas.create_line(start_x, base_y, end_x, base_y,
1622             ↪ fill=palette["surface_soft"], width=2)
1623         points: list[float] = []
1624         for index in range(180):
1625             t = -3.0 + 6.0 * index / 179
1626             x = start_x + (t + 3.0) / 6.0 * (end_x - start_x)
1627             y = base_y - 72 * pow(2.718281828, -(t * t))
1628             points.extend([x, y])
1629         self.canvas.create_line(*points, fill=palette["secondary"], width=3,
1630             ↪ smooth=True)
1631
1632     def _draw_canvas_background(self) -> None:
1633         palette = self.palette
1634         width, height = self._canvas_size()
1635         is_pink = self._is_pink_theme()
1636         self.canvas.delete("all")
1637         self.canvas.create_rectangle(0, 0, width, height, fill=palette["surface"],
1638             ↪ outline="")
1639         grid_color = "#FFE7F4" if is_pink else "#171717"
1640         for x in range(0, width, 42):
1641             self.canvas.create_line(x, 0, x, height, fill=grid_color)

```

```

1639     for y in range(0, height, 42):
1640         self.canvas.create_line(0, y, width, y, fill=grid_color)
1641
1642     if is_pink:
1643         rainbow_colors = ["#FF6DB8", "#FFD166", "#83E377", "#64C9FF", "#C99BFF"]
1644         for index, color in enumerate(rainbow_colors):
1645             inset = 24 + index * 10
1646             self.canvas.create_arc(
1647                 inset,
1648                 inset,
1649                 210 - inset / 3,
1650                 210 - inset / 3,
1651                 start=24,
1652                 extent=138,
1653                 outline=color,
1654                 width=6,
1655                 style="arc",
1656             )
1657
1658         self.canvas.create_oval(40, height - 84, 112, height - 42,
1659             ↪ fill="#FFC1E1", outline="")
1660         self.canvas.create_oval(92, height - 108, 136, height - 62,
1661             ↪ fill="#FFC1E1", outline="")
1662         self.canvas.create_polygon(124, height - 102, 138, height - 128, 142,
1663             ↪ height - 98, fill="#FFC1E1")
1664         self.canvas.create_arc(22, height - 106, 72, height - 38, start=90,
1665             ↪ extent=160, outline="#FF6DB8", width=7)
1666         self.canvas.create_line(58, height - 43, 52, height - 24,
1667             ↪ fill="#D86DA8", width=4)
1668         self.canvas.create_line(96, height - 43, 104, height - 24,
1669             ↪ fill="#D86DA8", width=4)
1670
1671         sparkle_points = ((width - 82, 54), (width - 42, 110), (width - 126,
1672             ↪ 148), (258, 46), (306, 96))
1673         for x, y in sparkle_points:
1674             self.canvas.create_text(x, y, text="*", fill=palette["accent"],
1675             ↪ font=("Segoe UI", 17, "bold"))
1676     else:
1677         self.canvas.create_text(
1678             width - 34,
1679             34,
1680             text="Ax=b",
1681             fill=palette["subtle"],
1682             anchor="e",
1683             font=("Consolas", 16, "bold"),
1684         )
1685     self.canvas.create_rectangle(14, 14, width - 14, height - 14,
1686         ↪ outline=palette["surface_soft"])
1687
1688     def _is_pink_theme(self) -> bool:
1689         return self.palette["bg"] == PINK_THEME["bg"]
1690
1691     def _redraw_idle_if_empty(self, _event: tk.Event) -> None:
1692         if not self.canvas.find_all():

```

```

1684         self.clear_canvas()
1685
1686     def _canvas_size(self) -> tuple[int, int]:
1687         return max(self.canvas.winfo_width(), 760), max(self.canvas.winfo_height(),
1688             ↪ 520)
1689
1690     def apply_theme(self, palette: dict[str, str]) -> None:
1691         super().apply_theme(palette)
1692         for card in (self.header_card, self.controls_card, self.art_card):
1693             card.configure(fg_color=palette["panel"],
1694                 ↪ border_color=palette["surface_soft"])
1695
1696         self.title_label.configure(text_color=palette["text"])
1697         self.subtitle_label.configure(text_color=palette["muted"])
1698         self.controls_title.configure(text_color=palette["text"])
1699         self.art_title.configure(text_color=palette["text"])
1700         self.status_label.configure(text_color=palette["muted"])
1701         self.note_label.configure(text_color=palette["muted"])
1702         self.canvas.configure(bg=palette["surface"])
1703
1704         self.effect_button.configure(
1705             fg_color=palette["accent"],
1706             hover_color=palette["accent_hover"],
1707             text_color=palette["accent_text"],
1708         )
1709         self.theme_button.configure(
1710             text="Повернути нормальну тему" if self._is_pink_theme() else "Рожевий
1711                 ↪ режим",
1712             fg_color=palette["segment_active"] if self._is_pink_theme() else
1713                 ↪ palette["segment"],
1714             hover_color=palette["segment_hover"],
1715             text_color=palette["text"],
1716         )
1717         for button in (self.gauss_button, self.music_button, self.clear_button):
1718             button.configure(
1719                 fg_color=palette["segment"],
1720                 hover_color=palette["segment_hover"],
1721                 text_color=palette["text"],
1722             )
1723
1724     class AlgorithmsApp(ctlk.CTk):
1725         def __init__(self) -> None:
1726             super().__init__()
1727             ctlk.set_appearance_mode("dark")
1728             ctlk.set_default_color_theme("blue")
1729
1730             self.title("Gauss Pivot Lab")
1731             self.geometry("1180x820")
1732             self.minsize(1080, 760)
1733
1734             self.current_page = "matrix"
1735             self.nav_buttons: dict[str, ctlk.CTkButton] = {}
1736             self.pages: dict[str, BasePage] = {}

```

```

1734 self.app_state = LinearSystemState()
1735 self.current_palette = DEFAULT_THEME
1736 self.pink_mode = False
1737
1738 self.fonts: dict[str, ctk.CTkFont] = {
1739     "title": ctk.CTkFont(family="Segoe UI", size=27, weight="bold"),
1740     "title_small": ctk.CTkFont(family="Segoe UI", size=17, weight="bold"),
1741     "subtitle": ctk.CTkFont(family="Segoe UI", size=13),
1742     "label": ctk.CTkFont(family="Segoe UI", size=13, weight="bold"),
1743     "body": ctk.CTkFont(family="Segoe UI", size=12),
1744     "button": ctk.CTkFont(family="Segoe UI", size=12, weight="bold"),
1745     "result": ctk.CTkFont(family="Consolas", size=15, weight="bold"),
1746     "metric": ctk.CTkFont(family="Consolas", size=16, weight="bold"),
1747     "nav": ctk.CTkFont(family="Segoe UI", size=12, weight="bold"),
1748     "mono": ctk.CTkFont(family="Consolas", size=13),
1749 }
1750
1751 self.page_configs: tuple[PageConfig, ...] = (
1752     PageConfig(
1753         page_id="matrix",
1754         nav_title="Система",
1755         title="Метод Гауса з вибором головного елемента",
1756         subtitle=(
1757             "Редугуй коефіцієнти, завантажуй JSON і розв'яжуй СЛАР з
1758             ↪ частковим "
1759             "вибором головного елемента для стійкішого прямого ходу."
1760         ),
1761     ),
1762     PageConfig(
1763         page_id="steps",
1764         nav_title="Хід методу",
1765         title="Прямий хід та pivoting",
1766         subtitle="Кожен крок показує вибраний головний елемент, перестановку
1767             ↪ рядків та занулення.",
1768     ),
1769     PageConfig(
1770         page_id="check",
1771         nav_title="Перевірка",
1772         title="Контроль точності",
1773         subtitle="Вектор розв'язку, нев'язки Ax - b і швидка візуальна
1774             ↪ оцінка похибки.",
1775     ),
1776     PageConfig(
1777         page_id="extra",
1778         nav_title="Додатково",
1779         title="Додаткові ефекти",
1780         subtitle="Окреме полотно для анімації, математичних частинок і
1781             ↪ портрета Гауса.",
1782     ),
1783 )
1784
1785 self.grid_columnconfigure(0, weight=1)
1786 self.grid_rowconfigure(1, weight=1)

```

```

1784         self._configure_treeview_style()
1785         self._build_topbar()
1786         self._build_pages()
1787         self._apply_theme()
1788         self.show_page(self.current_page)
1789         self.protocol("WM_DELETE_WINDOW", self._on_close)
1790
1791     def _build_topbar(self) -> None:
1792         self.topbar = ctk.CTkFrame(self, corner_radius=0)
1793         self.topbar.grid(row=0, column=0, sticky="ew")
1794         self.topbar.grid_columnconfigure(0, weight=1)
1795
1796         self.brand_label = ctk.CTkLabel(
1797             self.topbar,
1798             text="SLAE Solver",
1799             font=self.fonts["title_small"],
1800             anchor="w",
1801         )
1802         self.brand_label.grid(row=0, column=0, sticky="w", padx=20, pady=(18, 12))
1803
1804         self.nav_frame = ctk.CTkFrame(self.topbar, fg_color="transparent")
1805         self.nav_frame.grid(row=0, column=1, sticky="e", padx=20, pady=(18, 12))
1806
1807         for index, page in enumerate(self.page_configs):
1808             button = ctk.CTkButton(
1809                 self.nav_frame,
1810                 text=page.nav_title,
1811                 width=132,
1812                 height=36,
1813                 corner_radius=8,
1814                 font=self.fonts["nav"],
1815                 border_width=1,
1816                 command=lambda pid=page.page_id: self.show_page(pid),
1817             )
1818             button.grid(row=0, column=index, padx=(0, 8))
1819             self.nav_buttons[page.page_id] = button
1820
1821     def _build_pages(self) -> None:
1822         self.page_host = ctk.CTkFrame(self, corner_radius=0)
1823         self.page_host.grid(row=1, column=0, sticky="nsew", padx=20, pady=(0, 20))
1824         self.page_host.grid_columnconfigure(0, weight=1)
1825         self.page_host.grid_rowconfigure(0, weight=1)
1826
1827         matrix_page = MatrixPage(
1828             self.page_host,
1829             self.page_configs[0],
1830             self.fonts,
1831             self.app_state,
1832             self._notify_pages,
1833         )
1834         matrix_page.grid(row=0, column=0, sticky="nsew")
1835         self.pages["matrix"] = matrix_page
1836
1837         steps_page = StepsPage(

```

```

1838         self.page_host,
1839         self.page_configs[1],
1840         self.fonts,
1841         self.app_state,
1842     )
1843     steps_page.grid(row=0, column=0, sticky="nsew")
1844     self.pages["steps"] = steps_page
1845
1846     check_page = CheckPage(
1847         self.page_host,
1848         self.page_configs[2],
1849         self.fonts,
1850         self.app_state,
1851     )
1852     check_page.grid(row=0, column=0, sticky="nsew")
1853     self.pages["check"] = check_page
1854
1855     extra_page = ExtraPage(
1856         self.page_host,
1857         self.page_configs[3],
1858         self.fonts,
1859         self.app_state,
1860         self.toggle_pink_theme,
1861     )
1862     extra_page.grid(row=0, column=0, sticky="nsew")
1863     self.pages["extra"] = extra_page
1864
1865     def show_page(self, page_id: str) -> None:
1866         self.current_page = page_id
1867         self.pages[page_id].refresh()
1868         self.pages[page_id].tkraise()
1869         self._refresh_nav_buttons()
1870
1871     def _notify_pages(self) -> None:
1872         for page_id, page in self.pages.items():
1873             if page_id != self.current_page:
1874                 page.refresh()
1875
1876     def _on_close(self) -> None:
1877         for page in self.pages.values():
1878             stop_music = getattr(page, "_stop_music", None)
1879             if callable(stop_music):
1880                 stop_music()
1881         self.destroy()
1882
1883     def _refresh_nav_buttons(self) -> None:
1884         palette = self.current_palette
1885         for page_id, button in self.nav_buttons.items():
1886             if page_id == self.current_page:
1887                 button.configure(
1888                     fg_color=palette["segment_active"],
1889                     hover_color=palette["segment_active"],
1890                     text_color=palette["text"],
1891                     border_color=palette["segment_active"],

```

```

1892         )
1893     else:
1894         button.configure(
1895             fg_color=palette["segment"],
1896             hover_color=palette["segment_hover"],
1897             text_color=palette["muted"],
1898             border_color=palette["surface_soft"],
1899         )
1900
1901 def _configure_treeview_style(self) -> None:
1902     palette = self.current_palette
1903     style = ttk.Style()
1904     style.theme_use("default")
1905     style.configure(
1906         "Treeview",
1907         background=palette["surface"],
1908         foreground=palette["text"],
1909         fieldbackground=palette["surface"],
1910         rowheight=30,
1911         bordercolor=palette["surface_soft"],
1912         borderwidth=1,
1913         font=("Segoe UI", 10),
1914     )
1915     style.configure(
1916         "Treeview.Heading",
1917         background=palette["panel_alt"],
1918         foreground=palette["text"],
1919         relief="flat",
1920         font=("Segoe UI", 10, "bold"),
1921     )
1922     style.map(
1923         "Treeview",
1924         background=[("selected", palette["segment_active"])],
1925         foreground=[("selected", palette["text"])],
1926     )
1927
1928 def toggle_pink_theme(self) -> str:
1929     self.pink_mode = not self.pink_mode
1930     self.current_palette = PINK_THEME if self.pink_mode else DEFAULT_THEME
1931     self._apply_theme()
1932     if self.pink_mode:
1933         return "Рожевий режим увімкнено"
1934     return "Повернуто звичайну тему"
1935
1936 def _apply_theme(self) -> None:
1937     palette = self.current_palette
1938     ctk.set_appearance_mode("light" if self.pink_mode else "dark")
1939     self._configure_treeview_style()
1940     self.configure(fg_color=palette["bg"])
1941     self.topbar.configure(fg_color=palette["topbar"])
1942     self.brand_label.configure(text_color=palette["text"])
1943     self.page_host.configure(fg_color=palette["bg"])
1944
1945     for page in self.pages.values():

```

```
1946         page.apply_theme(palette)
1947         self._refresh_nav_buttons()
1948
1949
1950 def main() -> None:
1951     app = AlgorithmsApp()
1952     app.mainloop()
```

Файл src/__init__.py:

```
1 """Linear system solver application package."""
```

Висновок:

Під час виконання лабораторної роботи було розглянуто задачу розв'язання системи лінійних алгебраїчних рівнянь і реалізовано метод Гауса з вибором головного елемента. Використання часткового вибору головного елемента виявилось важливим для заданої системи, оскільки під час прямого ходу було виконано дві перестановки рядків. Це дозволило уникнути ділення на менш зручні за модулем елементи та підвищило чисельну стійкість алгоритму.

У результаті обчислень для системи варіанта було отримано вектор

$$x = (0.402501749943; 1.501308269656; 0.586163781582; -0.267825461892).$$

Після округлення до чотирьох знаків він збігається з контрольними значеннями: $x_1 = 0.4025$, $x_2 = 1.5013$, $x_3 = 0.5862$, $x_4 = -0.2678$. Визначник матриці дорівнює 18.93734589, тобто система має єдиний розв'язок.

Перевірка за нев'язкою показала, що

$$\max |Ax - b| = 2.220446049250 \cdot 10^{-16}.$$

Таке значення має порядок машинної похибки, тому знайдений розв'язок можна вважати чисельно точним для обчислень із подвійною точністю. Отже, реалізований алгоритм працює коректно, а результати розрахунків є достовірними.

Також було створено графічну програму з модульною структурою проєкту. У програмі реалізовано введення й редагування матриці, завантаження даних із JSON, перегляд кроків прямого ходу, контроль нев'язок, збереження звіту та додаткові елементи інтерфейсу. Розділення коду на окремі модулі спростило перевірку алгоритмічної частини та зробило програму зручнішою для подальшого розширення.